

MimirRAG: A Multi-Agent RAG Framework for Financial Data Retrieval with Metadata Integration

Magnus SamuelSEN^a, Wilmer Nyström^a, Somnath Mazumdar^a, Mansoor Hussain^b, Mikkel Strange^b

^aCopenhagen Business School, Solbjerg Plads 3, Frederiksberg, 2000, Denmark

^bJyske Bank, Copenhagen, Denmark

Abstract

Retrieval-augmented generation (RAG) systems offer a promising approach to reduce hallucinations and improve answer accuracy in large language models (LLMs), a requirement for reliable, financial analysis where answers must be grounded in verifiable evidence from filings rather than generated from model priors. However, designing RAG systems that extract meaningful insights from mixed financial documents and integrate into analyst workflows remains challenging. This paper introduces MimirRAG (Metadata-Integrated Multi-Agent Information Retrieval), a multi-agent RAG system developed iteratively to address these challenges. MimirRAG features a modular pipeline encompassing structure-preserving parsing of PDF filings, table-aware chunking, metadata extraction, agent-based retrieval with query planning and hybrid search, validation, and context-aware generation with numerical reasoning support. Our ablation study identifies three key technical enablers for effective financial RAG: metadata integration, table-aware chunking, and an agentic workflow. MimirRAG was evaluated quantitatively using FinanceBench and qualitatively through expert validation with four financial analysts. The system achieved 89.3% accuracy on FinanceBench, outperforming the original benchmark baselines. Expert feedback highlighted that successful deployment also requires calibrated trust, comprehensive data integration, and user personalization. We conclude that combining multi-agent RAG architecture with human-centric design principles can improve the extraction of meaningful insights in financial analysis.

Keywords: RAG, GPT, Financial Analysis, Agent, Metadata, Retrieval

1. Introduction

Large language models (LLMs)¹ have begun to transform workflows in finance. The financial sector, with its large volume of reports, filings, and other analyst-facing documents, stands to benefit from these advances [1]. However, standard LLMs in financial analysis carry the risk of *hallucinations*, unsupported or factually incorrect statements [2, 3]. In finance, even minor factual errors can have reputational, monetary, or legal consequences, making hallucination mitigation a key deployment challenge [1]. Furthermore, the volume of financial reports often leads to information overload, hindering analyst efficiency and prediction accuracy, especially for less experienced analysts [4]. Retrieval-augmented generation (RAG) offers a solution by connecting LLMs to external knowledge sources: RAG retrieves relevant evidence for a query and supplies it as context to the generator, constraining answers to retrieved, citable evidence rather

than relying solely on parametric memory [5, 6]. This focus on accuracy and traceability makes RAG attractive for knowledge-intensive domains like finance [3], with leading financial institutions developing RAG systems such as AskResearchGPT² to assist analysts in querying research report libraries.

Problem Context: Academia has increasingly turned its attention to customizing RAG systems for financial applications. Most existing work emphasizes technical performance improvements such as optimizing retrieval through enhanced embedding models [7], refining chunking strategies [8], or extending RAG architectures to better align with analyst workflows [9]. However, a critical dimension remains underexplored: the financial analysts' perspective. There is limited understanding of how these intended end-users perceive, interact with, and integrate such systems into their daily workflows. This paper addresses this gap by investigating both the technical design of RAG systems for finance and their human-centered evaluation.

Research Context: We focus on the strategic role of metadata. While its potential to enhance RAG perfor-

Email addresses: masa23ap@student.cbs.dk (Magnus SamuelSEN), winy23ab@student.cbs.dk (Wilmer Nyström), sma.digi@cbs.dk (Somnath Mazumdar), mahu@jyskebank.dk (Mansoor Hussain), mikkel.strange@jyskebank.dk (Mikkel Strange)

¹Generative Pre-trained Transformer (GPT) is a specific type of LLM developed by OpenAI.

²<https://www.morganstanley.com/press-releases/morganstanley-research-announces-askresearchgpt>

mance is acknowledged, systematic evaluation of metadata as a foundational component in financial RAG systems remains rare. Our primary research question is: **How can RAG-based solutions extract meaningful financial insights?** This is explored through three sub-questions: 1) How can a RAG-based solution be designed for financial information search tasks? 2) What design principles improve RAG systems’ precision and usability for financial analysts? 3) What key factors are crucial for integrating RAG-based solutions into financial analysts’ workflows?

To answer these we designed and prototyped *MimirRAG*, with a user interface (UI) for human evaluation. The target task is open-book financial question answering over corporate filings, including fact lookup, cross-period comparison, ratio calculation, and reasoning over tabular disclosures. Our corpus consists of PDF filings, mixed semi-structured documents containing free text, section hierarchies, and visually rendered tables. The system converts these into a structure-preserving intermediate representation before chunking and retrieval. Performance is assessed using FinanceBench [10], a benchmark tailored for open-book financial Q&A used in both academic [11, 12] and industry settings [13, 14, 15, 16]. This quantitative evaluation is supplemented by qualitative insights from four expert analysts. Our key contributions are:

1. We present a five-agent modular RAG architecture called *MimirRAG*, structured around pre-retrieval, retrieval, generation, and orchestration stages. It achieves 89.3% accuracy on FinanceBench through metadata integration and multi-agent design. Our ablation study demonstrates how structured document metadata (company name, report type, date) enhances retrieval precision.
2. Our investigation, combined with qualitative feedback from four financial analysts, reveals five design principles for improving precision and usability: *i)* domain-appropriate chunking strategies³, *ii)* metadata-driven targeted retrieval, *iii)* layered retrieval with granular filtering and validation, *iv)* calibrated trust through transparency and consistency, and *v)* integration with existing analyst data ecosystems and workflows.
3. We find that integrating RAG tools into financial analysts’ workflows depends on three factors beyond technical performance: *i)* calibrated trust, *ii)* comprehensive data integration, and *iii)* personalized augmentation of expertise.

2. Related Work

We survey the progression from early fine-tuned models such as FinBERT [17] to modular RAG pipelines that com-

bine chunking, metadata filtering, and agentic frameworks. We identify three gaps in current financial RAG research:

1) *Metadata*: Financial filings possess rich structured information (e.g., company, date, fiscal year, report type), yet systematic leverage of this metadata throughout the RAG pipeline remains partial or underexplored. 2) *Practical Insights*: Benchmark performance dominates the literature, qualitative feedback from financial analysts remains scarce. 3) *End-to-end Evaluation*: Comprehensive testing on realistic financial benchmarks assessing the complete retrieval-to-generation pipeline is less common, many studies focus on component-level optimizations or retrieval metrics alone.

2.1. LLMs in Finance

Early financial natural language processing (NLP) saw fine-tuned models like FinBERT enhance BERT by training on domain-specific texts. Subsequent developments include proprietary models like BloombergGPT [18] and open-source initiatives such as FinGPT [19] and the multi-agent FinRobot [20], all underscoring the value of domain adaptation. RAG systems extend this principle by grounding LLM responses in specific, externally retrieved contextual documents.

2.2. RAG in Financial Analysis

Research in financial RAG has rapidly evolved, exploring various pipeline enhancements (see Table 1 for a summary of key systems and their limitations). Initial efforts were focused to optimize core RAG components. For instance, studies have investigated the impact of different embedding models [7] or developed domain-specific chunking strategies, (such as element-based chunking for financial documents [8]). Others have explored advanced retrieval techniques including query expansion and re-ranking [12, 11], multi-document retrieval using semantic tagging and knowledge graphs (KGs) [21], or hybrid approaches combining vector search with KGs [22]. Some systems integrate RAG with chain-of-thought prompting and fine-tuning for tasks like stock prediction and question answering [23].

Despite these advancements, several challenges persist. Many approaches struggle with extracting and reasoning over structured data such as tables and performing numerical calculations [7, 12, 21], or they incur high computational costs [22, 24, 23, 11]. While the importance of metadata has been noted (e.g., for document selection [25] or filtering in specific contexts [3]), a systematic investigation into its deep integration and impact across the entire RAG pipeline is often missing, with some studies not testing its effect or explicitly excluding it [24, 12, 22]. Furthermore, as highlighted by Kim et al. [24] and evident in Table 1, comprehensive end-to-end evaluations incorporating human validation remain relatively uncommon.

³We use ‘document’ to refer to the source filing, and ‘chunk’ to refer to chunks retrieved by the retrieval system.

Table 1: Summary of RAG-based systems in financial applications

Paper	Focus Area	Methodology	Evaluation Method	Key Limitation
Iaroshev et al. [7]	Financial QA	• Testing various embeddings	• Context relevance • Faithfulness • Answer quality	• Poor on tables • Numerical reasoning weak • Format-specific issues
Kim al. [24]	Retrieval & End-to-end RAG	• Dense-sparse hybrid • Rerank	7 QA sets (incl. FinanceBench, FinDER); NDCG@10, RAGAS	• High compute cost • No metadata filtering
Lai et al. [25]	Financial QA Corpus; Multi-doc quantitative reasoning; Dynamic re-fresh.	• SEC-QA framework (dynamic QA gen.) • Program-of-thought RAG on financial filings.	• QA Accuracy & Retrieval metrics for multi-doc quantitative QA.	• Assumes data-doc mapping. • CodeGen higher compute cost.
Lee et al. [11]	Pre-retrieval & Reranking	• Query expansion • Long-context fusion	FinanceRAG metrics (accuracy, numerical reasoning)	• High computational cost • Manual prompt tuning
Li et al. [23]	QA + Stock Prediction	• CoT prompting • Real-time financial data	ARR, Accuracy (vs FinGPT)	• Computational expensive at large scales
Sarmah al. [22]	Hybrid RAG	• VectorRAG • GraphRAG • Metadata filtering	• Context precision • Faithfulness • Answer relevance	• High compute cost • Manual KG creation • Metadata effect not tested
Sarmah al. [3]	Reducing hallucination in financial report QA extraction.	• RAG (PaLM2, Llama2, etc.) • Metadata filtering for targeted retrieval • Chunking; MMR.	• BERTScore, BARTScore, Jaro.	• Metadata benefit is less clear for some metrics. • Outdated models.
Setty al. [12]	Retrieval Optimization	• Query expansion • Re-ranking • Chunking	FinanceBench (retrieval + QA metrics)	• Multi-doc still weak • Poor on structured data • No results on Metadata
Shah al. [21]	Multi-document RAG	• KG-RAG • Semantic tagging for multi-source QA	9 metrics incl.: • Faithfulness • Correctness	• Graph errors • Table/chart parsing weak • Tagging dependency
Taghvaei et al. [26]	Stock Price Prediction; Multi-modal RAG (10-K & news).	• LLM classifier with RAG for news (meta-data filtered). • Zero/few-shot.	• 3/6-month stock movement prediction (Weighted F1, MCC).	• Few-shot limited by prompt length/token limits.
Yepes al. [8]	Chunking Strategies	Element-based chunking (headers/tables)	FinanceBench Q/A performance	Not a full system test

2.3. Modular RAG

The trend towards modular RAG architectures decomposes the standard pipeline into specialized, interconnected components [6, 27]. In financial applications, these systems often adopt an agentic approach, aiming to replicate the analytical workflow of human analysts through collaborative AI agents [28, 20, 9] rather than serving solely as passive tools. For example, Chen et al. [28] utilized a mixture-of-agents framework with small, specialized LLMs for discrete tasks. Systems like FinRobot [20, 9] employ multiple agents, often with chain-of-thought prompting and RAG-based grounding, for complex tasks such as equity research. While promising for enhanced reasoning and flexibility, these agentic systems can be computationally intensive and often lack proper benchmarking or evaluation within real-world analyst workflows.

2.4. Contextualization

Our work addresses three limitations in financial RAG research. First, while prior studies incorporate metadata for filtering [3] or document selection [25], its role as a systematically integrated component across the full RAG pipeline remains underexplored [29]. We examine how metadata integration influences both retrieval performance and overall system design.

Second, existing research largely focuses on optimizing isolated components with limited emphasis on end-to-end evaluation under realistic conditions. We conduct an evaluation of MimirRAG on FinanceBench, complemented by expert validation, to derive design principles for improving precision and usability.

Third, factors influencing practical adoption, trust calibration, perceived usefulness, and workflow integration, remain insufficiently understood. By involving financial analysts in evaluation, we identify key determinants for integrating RAG systems into real-world workflows.

3. Methodology

This research employed a structured and iterative methodological approach, integrating the conceptual guidance of Saunders et al.’s Research Onion [30] with the procedural framework of Action Design Research (ADR) [31]. The Research Onion guided our high-level methodological choices, leading us to adopt a critical realist research philosophy, an abductive research approach, and a mixed-methods strategy. This involved a case study at a major Danish bank to understand analyst information workflows and an experimental strategy for the iterative development and refinement of the MimirRAG prototype.

The core of our research design was ADR, which conceptualizes the IT artifact as an *ensemble artifact* shaped by its organizational context and the learning that occurs through its use [31]. ADR facilitates generalizable design

knowledge through iterative cycles of Building, Intervention, and Evaluation (BIE) (refer to Figure 1), which are particularly beneficial for developing RAG systems where user interaction and contextual fit are important. Our research followed ADR’s main stages.

- *Problem Formulation (Stage 1)*: We initiated the research by identifying real-world analyst challenges (practice-inspired research) and designing an initial MimirRAG system based on established theories in RAG, information retrieval, and NLP (theory-ingrained artifact).
- *Building, Intervention, and Evaluation (BIE Cycles - Stage 2)*: MimirRAG was developed and iteratively refined through two BIE cycles. The artifact was deployed within the financial analysis environment of a major Danish bank. Four financial analysts interacted with the system (intervention), and their feedback directly informed its redesign (building), embodying the principle of reciprocal shaping. This involved mutually influential roles, with our team providing RAG expertise and analysts offering domain knowledge. Authentic and concurrent evaluation, integrating both benchmark testing and practitioner feedback, was continuous within each BIE cycle.
- *Reflection and Learning (Stage 3)*: Insights from benchmark performance (using FinanceBench) and expert validation were used to identify technical weaknesses (e.g., numerical reasoning, table parsing) and usability challenges, guiding priorities for subsequent development.
- *Formalization of Learning (Stage 4)*: The iterative process culminated in the derivation of practical design principles for financial RAG systems and theoretical insights into factors influencing their adoption (generalized outcomes).

This ADR process (Figure 1) facilitated a comprehensive evaluation of MimirRAG’s technical performance and real-world applicability, addressing our main research question. The design of the RAG solution for financial information search (first subresearch question) was primarily addressed during ADR Stages 1 & 2 through initial design, iterative development, and technical benchmarking. Design principles improving RAG precision and usability for financial analysts (second subresearch question) were formalized in ADR Stage 4, drawing from consolidated benchmark and analyst insights. Key factors for integrating RAG solutions into analysts’ workflows (third subresearch question) were identified through analyzing analyst feedback and usability from ADR Stages 2 & 3.

3.1. Analyst Feedback and Evaluation

A key ADR component, and addressing a noted gap in financial RAG research, was the deep involvement of end-

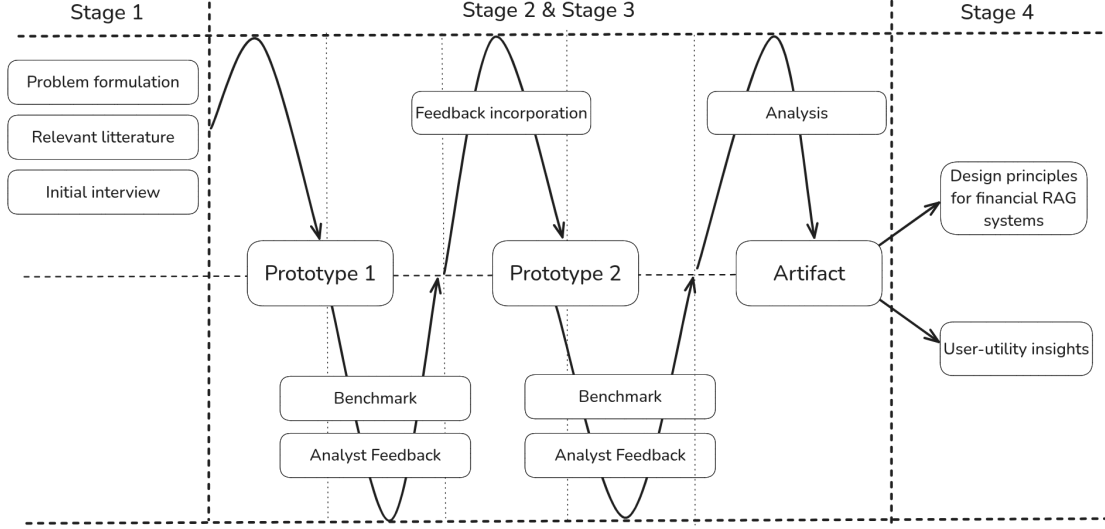


Figure 1: Iterative BIE process in MimirRAG development.

users. We conducted two iterative expert validation cycles with four financial analysts from a major Danish bank, as summarized in Table 2.

Participant 1 (P1), a senior sell-side analyst (Healthcare, more than 10 years experience), was central to Iteration 1. Initial scoping with P1 identified relevant documents (e.g., Novo Nordisk⁴ annual reports, SEC Form 20-F filings) and workflow pain points, informing the Iteration 1 knowledge base and interface design. After a two-week trial, P1 participated in a semi-structured interview focusing on usability, trust, and improvements. Iteration 2 incorporated P1’s feedback (notably private notes integration) and expanded evaluation to three additional analysts (P2: Director-level, P3: Assistant, P4: Senior) for diverse perspectives.

4. System Overview

MimirRAG (Metadata-Integrated Multi-Agent Information Retrieval) is a modular, agentic RAG workflow comprising five loosely coupled agents: Extractor, Planner, Search, Validator, and Writer. Each agent executes a distinct sub-task in the RAG pipeline and communicate via structured text-based outputs, enabling modularity and supporting independent adaptation or replacement. MimirRAG separates the pipeline into three logical stages: pre-retrieval, retrieval, and post-retrieval (see Figure 2) following best practices in RAG systems [6, 27].

Agentic decomposition provides several advantages. For example, failed retrievals can be dynamically retried

with adaptive query reformulation; individual agent logic, prompts, or models can be tuned or swapped in isolation; and heterogeneous models (e.g., lightweight search, heavyweight generation) can be efficiently composed. This modular architecture further facilitates rapid integration of new workflows, tools, or agent capabilities with minimal system-wide impact.

4.1. Pre-Retrieval Pipeline

The pre-retrieval stage transforms raw financial documents into a structured corpus with metadata and embeddings to enable hybrid-based (Best Match 25 (BM25)⁵ + vector) search. As illustrated in Figure 3, the pipeline includes: 1) robust document parsing and chunking, 2) structured metadata extraction, and 3) embedding and indexing in LanceDB.

4.1.1. Document Parsing and Chunking

Raw PDFs are processed using DOCLING [32]⁶, integrating DocLayNet [33] for layout analysis, TableFormer [34] for table extraction, and optical character recognition (OCR) as needed. Here, the PDF is the source delivery format of the filing, while the filing content itself contains both prose and visually rendered tables. We treat the corpus as mixed semi-structured documents. Docling converts each PDF into a structured intermediate representation and then into Markdown while preserving headings, section boundaries, and table content where possible. This structural preservation is particularly valuable in finance, given documented challenges with tables and numerical data [7, 12, 21]. OCR and table extraction are not error-free, especially for scanned or image-heavy tables, and can

⁴Novo Nordisk A/S is a Danish multinational pharmaceutical company <https://www.novonordisk.com/>

⁵It is a ranking function that extends TF-IDF (term frequency-inverse document frequency) by considering term frequency saturation and document length.

⁶<https://github.com/docling-project/docling>

Table 2: Overview of Expert Validation Activities and Participant Involvement Across Iterations

Evaluation Activity	Brief Description	Participants	Iteration
Scoping Interview	Understand P1 workflow & select Iter. 1 docs.	P1	Setup (Pre-Iter. 1)
In-depth Semi-Structured Interview	Assess baseline system: usability, trust, value.	P1	1
A/B Survey	Compare MimirRAG vs. Copilot outputs.	P1	1
Email Feedback on Notes Feature	Qualitative feedback on ‘search-my-notes’ feature.	P1	2
Semi-Structured Interviews	Broader qualitative feedback: usability, trust, improvements.	P2, P3, P4	2
Analyst Feedback Survey	Quantify perceptions (helpfulness, trust, time-saving) & gather open feedback.	P1, P2, P3, P4	2

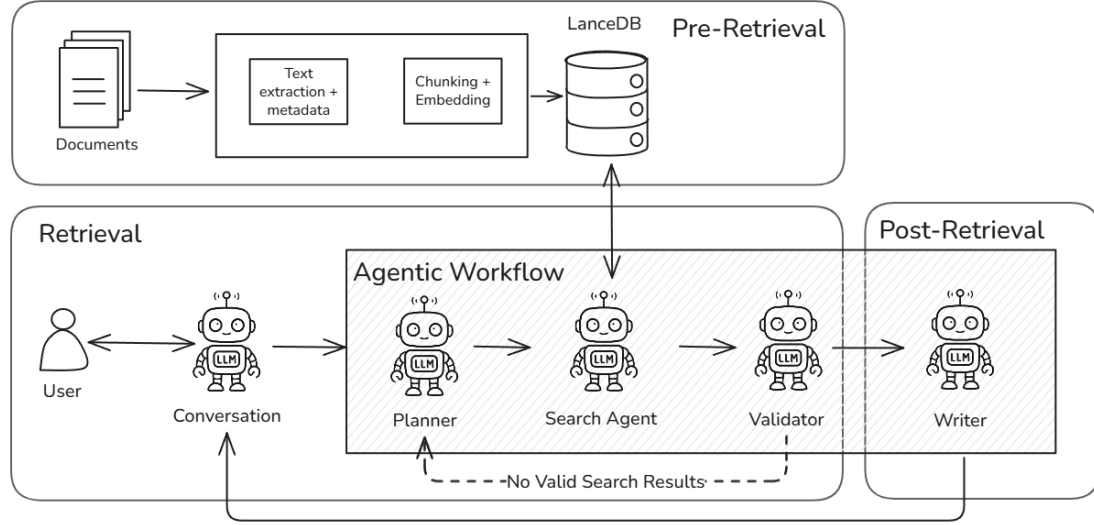


Figure 2: MimirRAG pipeline: pre-retrieval, retrieval, and post-retrieval stages.

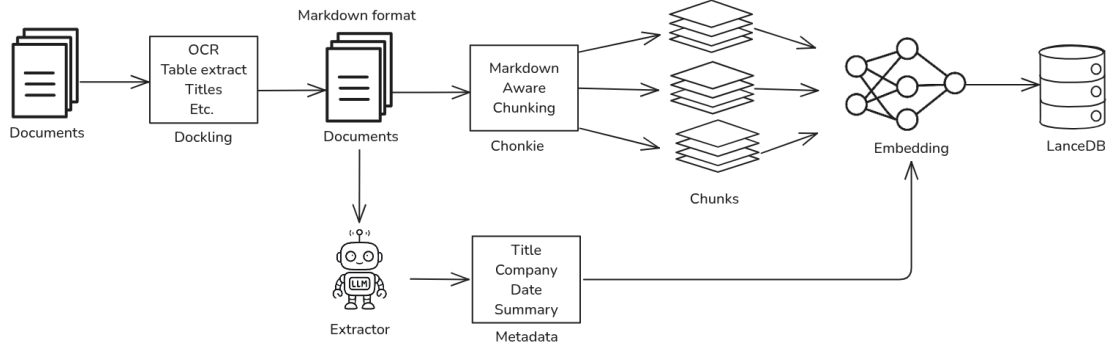


Figure 3: Pre-retrieval data pipeline representation

introduce segmentation or cell-boundary errors; our downstream chunking and validation steps are designed to mitigate, but not eliminate, these failure modes.

Documents are normalized to Markdown, then recursively chunked using CHONKIE⁷, splitting on headings, paragraphs, and whitespace, with fixed chunk and overlap sizes

($L_{\max} = 1800$, $\Delta = 300$), following industry best practices [35]. In this representation, tables are serialized as Markdown table blocks rather than flattened into plain text. To reduce errors caused by split tables, consecutive chunks are merged when one chunk ends with a table row and the next begins with a table row, up to a maximum merged length of 3600 characters. This table-aware chunking reduces cases where a retrieved chunk contains only a partial set of table rows needed for a financial calculation.

⁷<https://github.com/chonkie-inc/chonkie>

4.1.2. Extractor Agent

Metadata extraction by the *Extractor agent* leverages the first 1024 tokens (2–3 pages) of each parsed document to obtain essential metadata (see Table 3), namely title, company name, keywords, summary, date, and report type. These metadata are created by an LLM-based extraction step over the document prefix because these fields are typically available near the front matter of annual reports and filings. Extracted company names are standardized via local FTS (full text search) matching in LanceDB, with fallback lookups through SEC EDGAR⁸ and Danish central business register (CVR) APIs for robustness. Retrieved metadata is stored and indexed for later use in hierarchical retrieval.

Using the metadata taxonomy in Table 3, the Extractor agent improves document traceability and supports hierarchical retrieval by restricting the candidate document set before chunk-level search. In other words, metadata improves retrieval primarily by reducing the search space and thereby lowering semantic ambiguity during ranking. The same metadata are also passed forward as source context during final answer generation, as described in Section 4.4.2 and Section 4.5.1.

4.2. Embedding and Vector Storage

Document-level embeddings are generated from the summaries produced by the Extractor agent, while chunk-level embeddings are generated directly from the chunk text. `snowflake-arctic-embed-m v2.0` [36]⁹ embedding model (768 dimensions), is selected for its balance of embedding quality, inference speed, and strong performance on MTEB [37] and FinMTEB [38] benchmarks. These embeddings support semantic retrieval, while BM25 indexes over metadata and chunk text support sparse lexical retrieval; the combination enables the hybrid search setup used by the Search agent.

Embeddings and metadata are stored in LanceDB¹⁰, a serverless, embedded vector database. LanceDB utilizes the Lance open-source columnar format for efficient analytical queries and retrieval. In addition to vector search, LanceDB supports FTS indexes (BM25) on metadata fields (e.g., `title`, `summary`) and chunk content, which are crucial for the hybrid search capabilities detailed in Section 4.4.2.

4.3. Relational Document Linkage

Each chunk is linked to its source document using foreign key relationships (Figure 4), ensuring retrieved chunks can

be traced to their parent document. Document-level metadata and summaries extracted by the Extractor agent are stored with the parent document. This design supports explainability and our hierarchical retrieval approach.

4.4. Retrieval Pipeline

The retrieval stage begins once the user submits a query. Its goal is to return a concise, high-quality set of semantically aligned chunks with the query. Our system uses an agentic retrieval workflow including query decomposition, metadata filtering, hybrid search, and chunk validation.

4.4.1. Planner Agent (Query Decomposition)

The Planner agent initiates the retrieval process. Upon receiving the user’s natural-language query, its primary role is to decide whether the query can be searched directly or should be decomposed into simpler sub-questions. If decomposition is necessary, the agent identifies logical break points and generates a set of targeted sub-queries. For instance, a complex query like “How did Novo Nordisk’s profit change between 2022 and 2023?” would be decomposed into: 1) “What was Novo Nordisk’s profit in 2022?” 2) “What was Novo Nordisk’s profit in 2023?”, and 3) “What is the change in profit between 2022 and 2023?”. This decomposition allows for more focused subsequent searches and enables the system to handle comparative and multi-hop questions that require reasoning across multiple facts, aligning with plan → act patterns [39, 40, 41]. If the original query is already simple enough, it is passed through directly. The output of the Planner agent is a set of one or more search queries $\{q_i\}$ to be processed.

4.4.2. Search Agent

The *Search agent* executes sub-queries q_i from the Planner agent in parallel (up to 12 concurrent searches). It employs a hybrid search strategy that combines keyword-based (sparse) and semantic (dense) retrieval, augmented by structured metadata filters to narrow the search space before chunk retrieval. For each query, the agent derives: 1) a semantic query for vector similarity search, 2) keywords for FTS, and 3) metadata filters (e.g., company names, dates, report types). This is the main mechanism by which metadata improves retrieval: it programmatically excludes irrelevant reports before dense and sparse ranking are applied at the chunk level.

The available options for these metadata filters are dynamically populated at runtime by querying the vector database (cached with a one hour Time-To-Live (TTL) and refreshed upon new data additions), ensuring that agent-generated filter values are validated against the most current and valid set of choices. Agent-generated filters are validated programmatically against cached metadata using Pydantic [42] models integrated with Pydantic AI¹¹;

⁸The Electronic Data Gathering, Analysis, and Retrieval (EDGAR) system is used by the U.S. Securities and Exchange Commission (SEC) to collect, store, and provide public access to financial and operational filings submitted by companies and individuals.

⁹<https://huggingface.co/Snowflake/snowflake-arctic-embed-m-v2.0>

¹⁰<https://github.com/lancedb/lancedb>

¹¹<https://github.com/pydantic/pydantic-ai>

Table 3: Structured metadata extracted by the Extractor agent

Metadata	Description
Title	[Report Type] – [Year] ([Company Name])
Company Name	Standardized primary company name
Keywords	Up to five keywords capturing central financial topics, sectors, or strategic actions
Summary	Short summary of the input text
Date	Fiscal period-end date or publication date found in the document
Report Type	Specific document category (e.g., 10-K, Annual Report, Press Release)

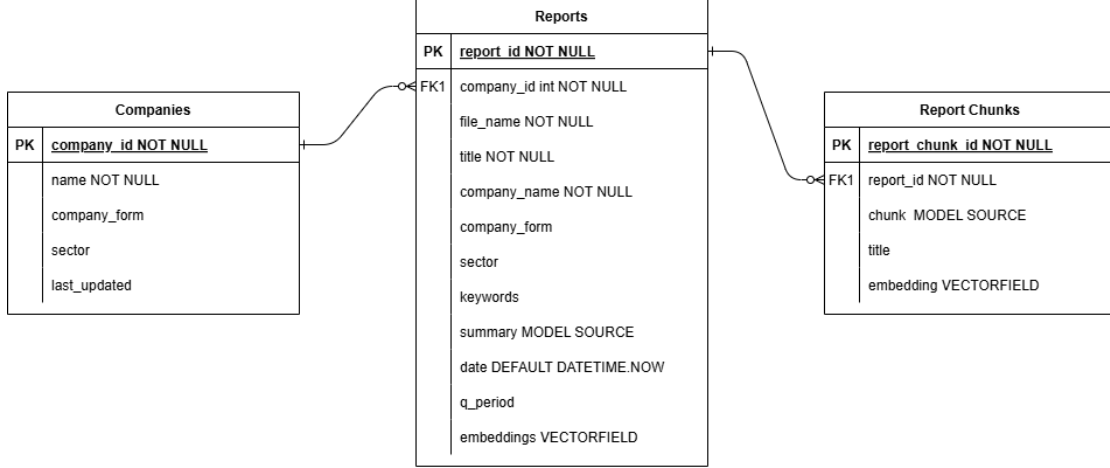


Figure 4: Entity-relationship (ER) diagram of vector database

validation failures trigger LLM-correction and retry cycles. The hierarchical search process (Figure 5) first applies these metadata filters to identify relevant reports. Chunks within these reports are then subjected to a hybrid search: dense retrieval (k-NN on vector embeddings) and sparse retrieval (FTS in LanceDB). Relevance scores are combined using Reciprocal Rank Fusion (RRF) to produce a top-K ranked list of chunks.

4.4.3. Validator Agent

The top-ranked chunks retrieved from a search are passed through the *Validator agent*, which determines if a chunk could contribute to answering the user’s question. We take advantage of parallel calls and process all chunks in one go. If no valid chunks are found, the system retries with reformulated queries (up to three attempts). If all attempts fail, the user is informed that no relevant content was found. We formalize the entire search pipeline in Algorithm 1, and provide a sequence diagram in Figure 7.

In Algorithm 1, Q denotes the original user query, S the set of sub-queries, m_s the metadata filters for sub-query s , k_s the lexical query terms, H_s the ranked search results, V_s the validated chunks, and A the accumulated validated set.

4.5. Post-Retrieval and Answer Generation

Post-retrieval organizes retrieved chunks and generates a final response grounded in the original documents.

4.5.1. Structured Input Construction

Once the validated chunks are retrieved, they are grouped and formatted into a structured input that serves as the context for the *Writer Agent*. This structured format is designed to improve both grounding and interpretability, inspired by a prompt format¹² that also aligns with findings from [43, 35]. Chunks are grouped by their originating report (`report_id`), and within each group, they are sorted by relevance and labeled as individual excerpts. This grouping ensures that the Writer agent can reason over coherent sets of information rather than isolated fragments. The final input to the Writer agent is a concatenation of these structured report sections, each enclosed in a `<source>` tag. An example of the format is represented in Figure 6. Users do not create this prompt structure manually; they submit natural-language questions, while the system constructs the internal retrieval and generation prompts automatically.

4.5.2. Writer Agent

The *Writer Agent* synthesizes its final response from four inputs: the original user query; grouped and validated chunks; document metadata (for source info); and a precision-and-citation prompt template. Answers include inline citations and a programmatically appended source list (refer to Figure 11).

¹²<https://blog.skypilot.co/deepseek-rag/>

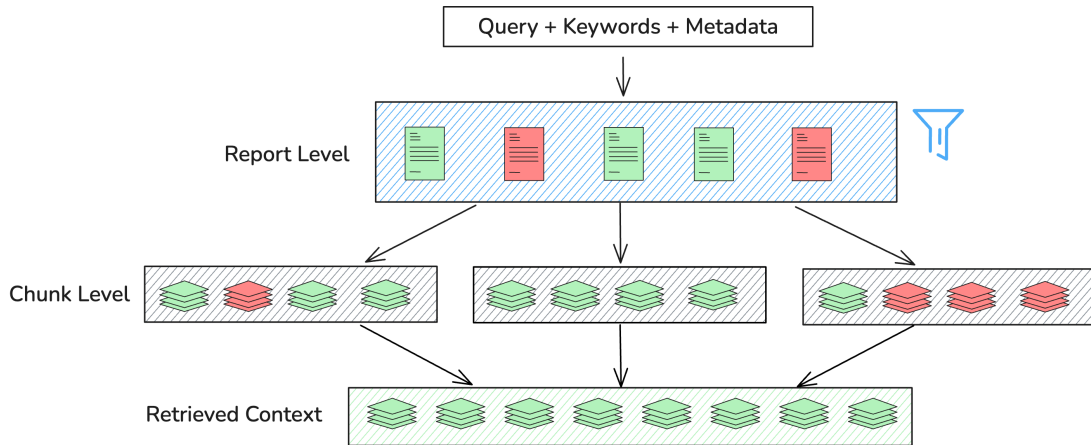


Figure 5: **Hybrid Search Process:** A query is first augmented with keywords and metadata filters. Relevant information is retrieved and used as pre-filters at the chunk level. A hybrid search is done over all relevant chunks and restricted to top-K

Structured Input Example

Original Query: “How did Novo Nordisk’s profit change between 2022 and 2023?”

Formatted Input to Writer Agent:

```
<source 1>: Annual Report 2023
Company: Novo Nordisk A/S
Summary: Highlights innovation, growth, and investments in 2023...
Date: 2023-12-31
Excerpt 1: OPERATING PROFIT AND MARGIN...
Excerpt 2: Financial statements of the parent company...
Excerpt 3: 4.1 Earnings per share...
</source 1>

<source 2>: Annual Report 2022
Company: Novo Nordisk A/S
Summary: Reports strong growth and demand for treatments in 2022...
Date: 2022-12-31
Excerpt 1: Total tax contribution...
Excerpt 2: Financial items (net) and tax...
Excerpt 3: FINANCIALS...
Excerpt 4: Operating profit increased by...
</source 2>
```

Figure 6: Sample of an input format

4.6. Agent Communication

MimirRAG’s inter-agent communication leverages Pydantic AI, a lightweight framework ensuring asynchronous operations and type-safe, structured interactions. Each agent is a self-contained class with an asynchronous `run` method, configured with task-specific prompts, tools (e.g., hybrid search), and Pydantic models for its inputs and outputs. While the overall workflow proceeds through logically sequential stages, the system utilizes asynchronous

execution to achieve parallel processing, particularly for performance-critical search and validation phases.

Figure 7 illustrates this *interaction logic*, showcasing how agents exchange structured messages. This architecture supports concurrent execution (parallel searches and validations) and Pydantic AI orchestration logic that handles fallbacks and retries. Data integrity is paramount, thus all agent outputs are validated against their Pydantic models. This strict validation, coupled with automatic retries

Algorithm 1 End-to-End Planning $\rightarrow$ Hybrid Retrieval $\rightarrow$ Validation Pipeline

Require: User query Q **Ensure:** Set A of validated chunks or **NO_ANSWER**

```
1:  $S \leftarrow \text{Planner}(Q)$   $\triangleright$  Generate initial sub-questions
2:  $A \leftarrow \emptyset$   $\triangleright$  All validated chunks
3:  $\text{round} \leftarrow 1$ 
4: while  $\text{round} \leq \text{MaxRounds}$  do
5:   for all  $s \in S$  (in parallel, up to 20) do
6:      $(k_s, m_s) \leftarrow \text{PrepareSearch}(s)$   $\triangleright$  Infer keywords
       and metadata filters
7:      $H_s \leftarrow \text{Search}(s, k_s, m_s)$   $\triangleright$  Metadata
       pre-filtering + hybrid retrieval + RRF
8:      $V_s \leftarrow \{h \in H_s \mid \text{Validate}(Q, s, h) = \text{true}\}$ 
9:     emit  $V_s$   $\triangleright$  Thread-local output
10:   end for
11:    $R \leftarrow \bigcup_{s \in S} V_s$ 
12:    $A \leftarrow A \cup R$ 
13:   if  $|A| \geq 1$  then
14:     return  $A$ 
15:   end if
16:   if  $\text{round} = \text{MaxRounds}$  then
17:     break
18:   end if
19:    $S \leftarrow \text{Replan}(Q, S)$   $\triangleright$  Generate new sub-questions
20:    $\text{round} \leftarrow \text{round} + 1$ 
21: end while
22: return NO_ANSWER if  $A = \emptyset$  else  $A$ 
```

(three by default) on failure, ensures reliability, simplifies error handling, and aids system maintenance.

4.7. Conversation Agent

While the five agents constitute the core RAG workflow for processing and answering queries from scratch, MimirRAG incorporates an additional *Conversation agent* that acts as an intelligent routing layer, managed by the **ConversationService** on the backend. For the Conversation agent, the RAG workflow is a tool that is used to answer the user’s question. The primary role of the Conversation agent is to manage the dialogue flow between the user and the RAG workflow, particularly for follow-up questions within an ongoing conversation. When a user submits a message, the **ConversationService** first determines if it’s the initial query of a conversation or a subsequent one. For the first message, the query is directly routed to the full RAG workflow, starting with the Planner agent, to retrieve and synthesize an answer. For subsequent messages, the Conversation agent is invoked. It analyzes the new user message in the context of the preceding conversation history. Based on this analysis, and guided by its specific prompt, it decides whether the new message requires a full, new retrieval process (e.g., the user asks a completely new question) or if it can be directly answered based on already retrieved content.

This approach makes user interaction more natural and efficient by avoiding redundant RAG workflow executions for conversational turns that do not necessitate new information retrieval. The Conversation agent thus enhances the user experience by providing more context-aware and responsive interactions. This also makes the system more modular and easier to extend with new features, as they can simply be added as new tools to the Conversation agent.

4.8. Prompt Engineering

Each agent uses a task-specific prompt. Prompt templates remain static across user sessions, while the content is dynamically filled based on the query- and chunk-specific content. Users interact with the system through a standard chat/query interface and provide only the natural-language question. They do not generate the internal prompts shown in Figure 8. By using prompt templates, we ensure consistency across the RAG workflow and make it easier to fine-tune individual stages independently. In the initial development phase, our priority was to establish a functional end-to-end workflow for each agent. Consequently, we focused on composing clear and straightforward system- and user-prompts that accurately described each agent’s intended task, enabling the required function calls and basic output formatting. While no systematic A/B testing was conducted, prompts were iteratively refined on an ad-hoc basis in response to observed errors or unexpected behavior during development and user interaction. For instance, the Search agent’s prompt was updated to default to searching within the last 12 months if no specific date was provided by the user, a change driven by observing common user interaction patterns.

Our approach to prompt design aims to enhance agent reliability and performance. It is guided by [44, 45]: ensuring the agent is aware of its operational environment and available tools, clearly documenting tool functionalities from the agent’s perspective, and designing instructions by considering what information and guidance an agent would need to perform its task effectively. The overall prompt template structure used is shown in Figure 8.

4.9. Benchmarking

We employ FinanceBench, a curated dataset tailored for assessing financial question-answering systems. It comprises over 10,000 question-answer pairs derived from 361 public filings released between 2015 and 2023, specifically designed to benchmark RAG systems and LLMs in financial contexts. Each entry includes a question, a gold-standard answer, and annotated evidence text, grounding the response in a specific location within a filing. This structure makes FinanceBench ideal for evaluating both retrieval accuracy and reasoning capabilities. FinanceBench tests two core competencies. They are 1) Retrieval: The ability to locate relevant evidence within

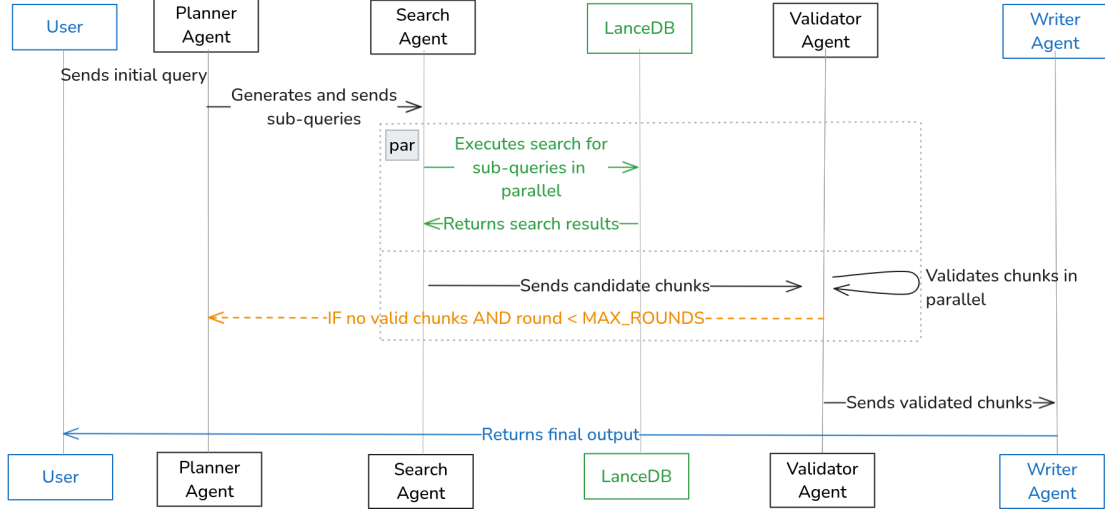


Figure 7: Sequence diagram illustrates the interaction between agents in the hierarchical retrieval and validation pipeline. Green arrows denote the parallel search against the vector store, black arrows show data flow between agents, the dashed orange arrow represents the fallback to replanning when no chunk is validated and $\text{round} < \text{MAX_ROUNDS}$, and the blue line denote the final response back to the user.

Agent Prompt Structure

```

# Task: {name of task}
{Overall explanation of task and environment.}
## Instructions:
{Step by step instructions.}
## Input Data:
Data from user, agent or search. For example:
<user_query> query </user_query>

```

Figure 8: Sample of an agent prompt structure

a realistic corpus of SEC filings and 2) Reasoning: The capacity to perform logical and numerical reasoning over retrieved evidence to address analyst-style questions. The dataset includes fact-based, reasoning-based, and context-aware questions, mirroring typical financial analyst research questions.

FinanceBench features three primary question categories. 1) Domain-relevant: A fixed set of 25 common analyst checks, e.g., *Has Company X paid a dividend in fiscal year Y?* 2) Novel-generated: Analyst-crafted questions simulating realistic variations of typical Q&A. 3) Metric-generated: Programmatically generated from 18 baseline items in financial statements, including derivatives. Sub-categories based on reasoning types, numerical reasoning, information extraction, and logical reasoning, enable detailed performance analysis.

4.9.1. Dataset Subset

We use the 150-question open-source evaluation subset released by the authors for their original experiments, as

the full FinanceBench dataset is not publicly available. This subset comprises: 50 domain-relevant questions, 50 novel analyst-generated questions, and 50 metric-based questions derived from financial statements, covering 32 companies. This combination ensures a balanced assessment of direct information retrieval and nuanced reasoning tasks.

4.9.2. Answer Checking

The benchmark questions are evaluated against the gold-standard answers which is performed by a Judge Agent, an LLM-based evaluator (specifically, GPT-4o in our setup). The Judge Agent categorizes each answer into three labels. 1) *Correct* means the answer matches the gold label and is grounded in the correct evidence. Minor discrepancies (e.g., small rounding errors) are tolerated; 2) *Incorrect* means the answer is factually wrong, contains calculation errors, or contradicts the gold answer; 3) *Failure to Answer* The model explicitly indicates it cannot answer the question. This aligns with the evaluation criteria used in the original FinanceBench paper, where they used human evaluators to score the answers. In addition to the label, the Judge Agent is also prompted to provide an explanation of the label.

4.9.3. Retrieval

In addition to evaluating the final generated answer, we also assess the performance of the retrieval component using several metrics:

- **Hit Rate (Hit@k):** This metric measures the percentage of questions for which the correct *document* containing the grounding evidence (as defined by FinanceBench) is found within the top k retrieved documents. We specifically report Hit@1 and Hit@5, indicating whether the document containing the correct

evidence was ranked as the first or within the top five retrieved documents, respectively. This assesses the retrieval system’s ability to identify and rank the most relevant source documents.

- **Chunk Similarity:** To evaluate the quality of the retrieved chunks at a content level, for each question, we identify the single retrieved chunk (from the top-k set) that has the highest similarity to the gold-standard evidence. These individual maximum similarity scores are then averaged across all 150 questions in the benchmark dataset. We provide two different similarity metrics:

- *Cosine Similarity:* Calculated on the text embeddings (‘snowflake-arctic-embed-m-v2.0’), this measures the semantic similarity between the retrieved chunk and the gold evidence. Higher values (close to one) indicate greater semantic relevance.
- *Levenshtein Similarity:* This metric quantifies the textual similarity based on the minimum number of single-character edits (insertions, deletions, or substitutions) required to change one text into the other. We report a normalized similarity score where higher values indicate greater textual overlap with the gold evidence. We remove all markdown formatting from the retrieved chunks before calculating the similarity, as the evidence text is formatted as plain text.

These retrieval metrics provide insights into the effectiveness of the system’s ability to locate and rank the precise information needed to answer the user’s query, complementing the end-to-end evaluation performed by the Judge Agent.

4.9.4. Existing Benchmarks and Retrieval Strategies

Table 4 defines the retrieval strategies evaluated on FinanceBench, from the original paper and other works. Crucially, to establish an updated oracle baseline with one of our evaluated LLMs, we ran the oracle evidence pages (perfect retrieval) with GPT-4.1 as the generator. This configuration achieved 93.3% accuracy using our Judge Agent evaluation, providing an updated reference point for what is achievable with perfect context and current models. Table 5 presents performance metrics for various models and strategies on the 150-question subset.

Databricks [13] achieved ~75% accuracy using o1-preview with long-context RAG, leveraging a 64K-token context window to process top-125 filing chunks. OODA [46] employed an Observe-Orient-Decide-Act (OODA) loop with retrieval, reaching 82% accuracy with GPT-3.5-turbo, surpassing their fine-tuned retrieval approach. They removed 9 questions due to broken or missing document sources,

deeming 141 of 150 questions suitable for benchmarking, though specific details about the disregarded questions are not provided.

Mafin 2.5 [16]: Vectify AI’s system achieves 98.7% accuracy with GPT-4o, built on PageIndex, a reasoning-based RAG framework that replaces vector similarity with hierarchical tree navigation over document structure. PageIndex constructs a semantic tree from document headers and LLM-generated summaries, enabling structured reasoning during retrieval rather than embedding-based matching. An earlier version of the system documentation also described reinforcement learning with Monte Carlo Tree Search (MCTS) as part of the pipeline¹³, though the full implementation details have not been published. Mafin adjusts 12 of the 150 benchmark questions: 6 removed as benchmark errors, 5 corrected for multiple valid answers (MVA), and 1 for same evidence, different conclusion (SEDC). These adjustments are disclosed¹⁴. Under the original unadjusted gold standards, their accuracy would be approximately 90.7% (136/150), close to our Oracle baseline. Given the limited methodological transparency, Mafin 2.5 is excluded from direct replicable comparison with MimirRAG. With the exception of [13], neither OODA nor Mafin 2.5 disclose prompt templates or the number of reasoning rounds used during benchmarking, limiting direct comparability. We find four key takeaways for RAG system development based on these previous results:

- *Value of Retrieval:* Basic RAG approaches demonstrate substantial gains over closed-book models. For instance, simply providing the full filing context boosted accuracy from 9% (GPT-4-Turbo, closed-book) to 79%, and long-context RAG systems like Databricks achieve ~75%. This underscores that access to relevant information is a primary driver of performance.
- *Better reasoning:* Once retrieval is in place, employing agentic loops like OODA (82% accuracy), significantly improves results, bringing performance closer to the 85% oracle baseline. This suggests that refining *how* information is identified, selected and reasoned about is crucial.
- *Improved base model:* Our new oracle baseline (93.3%), highlights that as LLMs become more capable, the performance ceiling for RAG systems, even with ideal retrieval, inherently rises.
- *Beyond the Oracle with Advanced Architectures:*

¹³Per the Mafin 2.5 GitHub repository prior to its October 2025 revision; current documentation describes the system solely as reasoning-based RAG powered by PageIndex.

¹⁴https://github.com/VectifyAI/Mafin2.5-FinanceBench/tree/main/human_evaluations

Table 4: Retrieval strategies evaluated on FinanceBench.

Retrieval Strategy	Description
Closed book	Model relies solely on pre-trained knowledge, without external context or documents.
Shared vector store	All filings indexed in a single vector database; retrieves top-k passages across all documents.
Single vector store	Each filing has its own vector index; retrieval is scoped to the relevant filing.
Full filing in context	Entire filing text (up to the model’s token limit) is provided as in-context input.
Oracle	Supplies only the exact "gold-standard" evidence pages containing the answer.
Mafin 2.5	Reasoning-based RAG via PageIndex: hierarchical tree navigation over document structure; replaces vector similarity with LLM-driven structured retrieval.
OODA loop	Iterative cycle: Observe, Orient, Decide, Act; applied to retrieval and reasoning.

Table 5: Performance summary on the 150-question FinanceBench benchmark. Results marked with * are from [10].

Retrieval Strategy	Context Width	Model	Accuracy (%)
Closed book*	–	GPT-4-Turbo	9%
Shared vector store*	–	GPT-4-Turbo	19%
Single vector store per filing*	–	GPT-4-Turbo	50%
Full filing in context*	95K tokens	GPT-4-Turbo	79%
Oracle (evidence pages)*	Evidence pages	GPT-4-Turbo	85%
Our Oracle (evidence pages)	Evidence pages	GPT-4.1	93.3%
Databricks	64K tokens	o1-preview-2024	~75%
OODA	–	GPT-3.5-turbo	82%
Mafin 2.5 (Vectify AI)	–	GPT-4o	98.7%

State-of-the-art pipelines like Mafin 2.5 (98.7% accuracy) demonstrate that it’s possible to surpass the traditional oracle. This is achieved not only through more capable LLMs (GPT-4o vs. GPT-4-Turbo in the original oracle) but also through structured, reasoning-based retrieval that preserves document hierarchy, rather than relying on flat embedding similarity. This implies that the retrieval ceiling can be raised by rethinking how documents are indexed and navigated, not solely by improving embedding or generation models.

We aim to assess MimirRAG’s performance relative to replicable established benchmarks, such as the original FinanceBench paper’s results and our own updated Oracle baseline. Our evaluation leverages the entire 150-question FinanceBench subset, and we will discuss potential inherent errors in the benchmark.

5. Experimental Analysis

Benchmarking provides crucial insights into a system’s technical performance under controlled conditions.

5.1. Selected LLM Variants

We evaluated our pipeline using OpenAI’s GPT-4 language model available at the partner company, chosen to explore trade-offs between model size, response quality, and application programming interface (API) cost. GPT-4.1 was our upper-bound baseline for language generation and instruction following. GPT-4.1 Mini is a distilled variant offering a balance of quality and cost, while GPT-o4 Mini is a reasoning-optimized variant. Table 6 details

these models where MMLU score refers to a model’s performance on the massive multitask language understanding (MMLU) benchmark. We calculated the relative cost per % accuracy of each model based on the cost of the input and output tokens divided by the accuracy of the model¹⁵

$$\text{Relative Cost} = \frac{\text{Input Price} + \text{Output Price}}{\text{Accuracy}}$$

5.2. Environment Setup

The system is deployed using Azure Container Apps, with the API running as the primary container and both the UI and proxy deployed as sidecars within the same app. The containers use different base images: Debian 12 (Slim Bookworm) for the API and UI, and Alpine for the proxy. Resource allocations are defined in Azure as follows: API: 0.75 vCPU and 2.6 GB RAM; UI: 0.5 vCPU and 0.7 GB RAM and Proxy: 0.25 vCPU and 0.5 GB RAM. A NVIDIA Tesla T4 (with 2,560 CUDA cores, 320 tensor cores and 16 GB of GDDR6 GPU memory) is provisioned on demand when GPU acceleration is required. For the first iteration, all experiments are run with max rounds set to three and max retrieved reports set to five. Concurrency is adjusted based on model and rate limits (Algorithm 1). Temperature for all agents is set to 0.1, except for o4-mini, which at the time of testing did not accept a temperature setting.

Table 6: Comparison of LLMs used in experiments.

Model	Context Length	MMLU Score	Pricing (USD per 1M tokens)	Release
GPT-4o	128K tokens	87.0%	\$2.50 (input) / \$10.00 (output)	May 2024
GPT-4.1	1M tokens	90.2%	\$2.00 (input) / \$8.00 (output)	Apr 2025
GPT-4.1-mini	1M tokens	87.5%	\$0.40 (input) / \$1.60 (output)	Apr 2025
GPT-o4-mini	200K tokens	?%	\$1.10 (input) / \$4.40 (output)	Apr 2025

Note: Pricing information from OpenAI API documentation is subject to change.

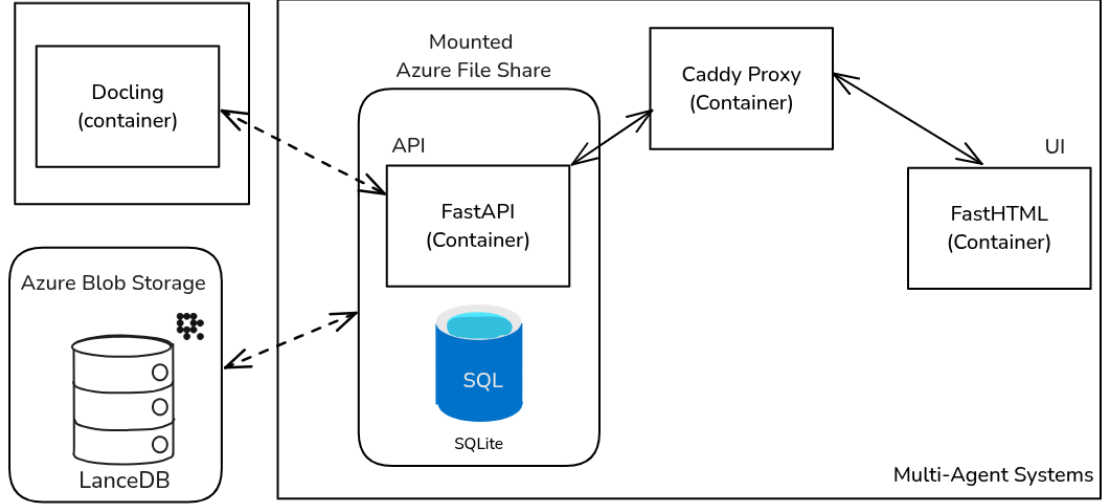


Figure 9: High-level Azure deployment diagram for MimirRAG.

5.3. Prototyping

MimirRAG was prototyped as a web application deployed on Azure Container Apps (see Figure 9). The backend, built with FastAPI¹⁶ for asynchronous processing, interfaces with LanceDB (in Azure Blob Storage) for vector/metadata storage and SQLite (on Azure Files) for relational data. The UI enabled users to submit natural-language queries, manage conversational history, and dynamically update the knowledge base by uploading documents and managing company profiles. Internal prompts for the agents were predefined by the system and not manually authored by end-users. Evaluation primarily focused on the chat interface (refer to bottom of Figure 10).

Figure 10 displays three of the five primary webpages of MimirRAG (the remaining two are omitted due to space constraints). The top section illustrates the landing page, the middle section shows the query interface, and the bottom section presents the main chat interface.

5.4. Ablation Study

The study establishes a baseline using Naive RAG (with a shared vector store) and a custom chunking and prompt setup, ensuring it reflects updated embedding and LLM models. It then performs ablations to assess the effects

of various LLM variants and retrieval configurations (see Table 7). We focused our ablation studies on the GPT-4.1-mini model, as it was the cheapest model, allowing us to test many configurations. The first iteration focused on establishing a baseline of our system and identifying initial performance characteristics. The results from these configurations informed the design of iteration two. These settings constitute the main ablation study for the paper (refer Table 7). They isolate the impact of metadata filtering, individual retrieval-stage agents, search breadth, and LLM choice on the end-to-end system.

5.5. First Iteration

5.5.1. Performance of Models

Table 8 summarizes the performance across different models and retrieval configurations in Iteration 1. Our reimplemented Naive RAG setup, using GPT-4.1-mini with our enhanced preprocessing (Docling for PDF to Markdown conversion and Markdown-aware chunking) and snowflake-arctic-embed-m-v2.0 embeddings, achieved 65.3% accuracy. This significantly outperforms the 19% baseline reported in the original FinanceBench paper for their GPT-4-Turbo Naive RAG configuration. This substantial improvement underscores the collective impact of more advanced LLMs, better embedding models, and a more robust document preprocessing and chunking pipeline than what was likely employed in the original study. Importantly, we do this with a small top-k of 10, compared to

¹⁵It does not account for potential differences in token usage between models.

¹⁶<https://fastapi.tiangolo.com/>

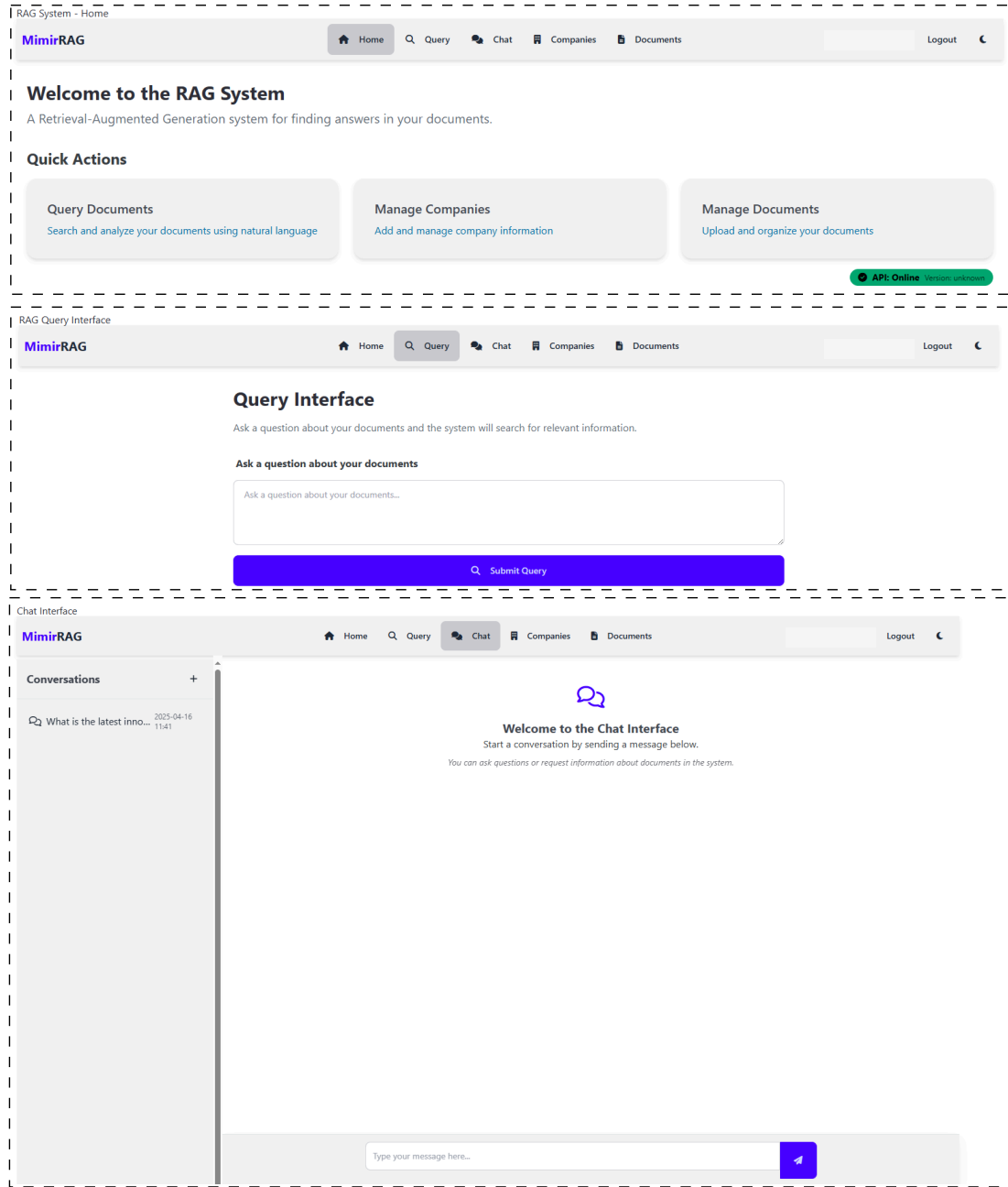


Figure 10: Screenshots of MimirRAG prototype: home page (top), query interface (middle) and chat interface (bottom)

Table 7: Ablation settings for iteration 1

Configuration	Description
Naive RAG	Re-implementing the original Shared Vector Store RAG system from the FinanceBench paper.
Retrieval agents impact	Testing the effect of removing the Planner, Search, or Validator agent individually from the MimirRAG setup.
Metadata filtering	Comparing performance with and without metadata constraints during search.
LLM variant	Evaluating GPT-4.1, GPT-4.1 Mini, and GPT-o4 Mini.
No. of Chunks (Search Breadth)	Varying the number of retrieved chunks per sub-question ($k=10$ vs. $k=30$).

previously discussed results by Databricks using top-128 ($\sim 75\%$).

The full MimirRAG system, in its best Iteration 1 configuration (GPT-4.1, Mimir, 10), achieved 82.7% accuracy. This result approached the original FinanceBench Oracle performance (85% with GPT-4-Turbo) and demonstrates strong capability even before the targeted optimizations introduced in Iteration 2. In terms of relative cost, the full MimirRAG configuration with GPT-4.1-mini (Mimir, 10) was the most cost-effective among the MimirRAG setups, with a calculated relative cost of \$0.026 per % accuracy. The full GPT-4.1 model in the same configuration was approximately 4.65 times more expensive per percentage point of accuracy. The qualitative difference between the Naive RAG approach and the MimirRAG system is illustrated in Figure 11. It presents an example where the Naive RAG system (left side of Figure 11) struggles with document disambiguation for a question about Verizon’s Quick Ratio, retrieving information from three different documents and failing to find the necessary data. In contrast, the MimirRAG system (right side of Figure 11), even with the cost-effective GPT-4.1-mini model, correctly identifies the relevant Verizon annual report, extracts the necessary figures, and accurately calculates the Quick Ratio. This highlights MimirRAG’s improved ability to precisely locate and utilize correct information from a larger corpus, a key advantage of its structured agentic approach and metadata utilization over simpler RAG techniques.

5.5.2. Retrieval Analysis

Table 9 presents document-level retrieval metrics. Metadata filtering showed a clear advantage. For instance, GPT-4.1-mini (Mimir, 10) achieved Hit@1 of 0.65 and Hit@5 of 0.81 compared to 0.53 and 0.73 for the No Meta configuration. Compared to the Naive RAG configuration, we observed large gains in Hit@1 while the gap is smaller at Hit@5. Metadata also led to a large reduction in the average number of documents retrieved, and the Validator agent effectively reduced the number of chunks retrieved. This supports the proposed mechanism that metadata improves answer quality mainly by restricting the candidate document set before chunk-level retrieval, thereby reducing ambiguity and irrelevant evidence.

Figure 12 provides a deeper insight into the quality of the retrieved content by displaying the average maximum chunk similarity scores. The analysis reveals several key trends. The GPT-4.1 (10) configuration (full MimirRAG), which also achieved the highest end-to-end accuracy in Iteration 1 (82.7%), correspondingly exhibited the highest chunk similarity scores on both metrics (Cosine 0.80, Levenshtein 0.59). This underscores the importance of retrieving semantically and textually aligned chunks for generating correct final answers.

Furthermore, comparing MimirRAG configurations

against Naive RAG variants using GPT-4.1-mini (Bar 2, 3, and 6 in Figure 12) reveals an interesting pattern. The GPT-4.1-mini (10, Naive RAG, Planner) setup (Bar 3) achieved the highest average maximum Cosine similarity (0.78) among these GPT-4.1-mini based configurations, surpassing even the full MimirRAG GPT-4.1-mini (Mimir, 10) system (Bar 6: Cosine 0.74) on this specific retrieval metric. Despite this superior chunk similarity, the full MimirRAG system delivered significantly better end-to-end accuracy (76.0% vs. 69.3% for ‘Naive RAG + Planner’ and 65.3% for basic ‘Naive RAG’). We speculate that while the Planner in a simplified Naive RAG context might effectively identify semantically similar content, this content may not always be optimally relevant or from the precisely correct source. For example, a highly similar passage from a prior year’s report rather than the target year.

5.5.3. Error Analysis by Reasoning Type

An error analysis of the best-performing configuration (GPT-4.1, Mimir, 10) revealed that ‘Unspecified’ (novel-generated) questions had the most incorrect answers (Figure 13). We manually checked each error and found that many were due to numerical errors, but also related to partly missing data. This was often due to tables that had been split across chunks. If only the first part of the table was in the context, the Writer agent would try to answer the question based on that partial table, and often make estimates on the missing data. This means that some of the numerical errors are due to missing data, and not necessarily a numerical reasoning error, but rather an information retrieval problem. However, there were also errors due to direct numerical reasoning errors. A small number of errors were also due to poorly specified questions in the benchmark, for example: *financebench_id: 00601*: “What drove the reduction in SG&A expense as a percent of net sales in FY2023?” Missing the relevant company name, this is impossible with a shared vector store. *financebench_id: 00822*: “Were there any board member nominees who had substantially more votes against joining than the other nominees?” Missing both the company name and the relevant year.

5.5.4. Lessons Learned from first Iteration

Iteration 1 revealed several key insights that directly informed refinements in Iteration 2: 1) model size mattered, with GPT-4.1 generally outperforming smaller variants; 2) metadata improved performance through better precision in hierarchical search; 3) a low ‘Failed to Answer’ rate suggested effective data retrieval in most cases; 4) there were diminishing returns for chunk count initially, as scaling from 10 to 30 chunks for GPT-4.1-mini offered little accuracy gain without better ranking; 5) numerical reasoning emerged as a key challenge; and 6) table chunking issues, where split tables led to incomplete context for the Writer agent, were identified.

Table 8: Iteration 1: Performance comparison of LLMs and configurations. Results marked with * are from [10]. ** GPT-4.1-mini is used as the Search and Validator Agent in this setting for improved latency (this model is cheaper but not accounted for in the cost column).

Model	Configuration	Correct	Incorrect	Failed to answer	Rel. Cost (\$/% Acc.)
GPT-4-turbo*	Naive RAG*	29 (19%)	20 (13%)	101 (68%)	N/A
GPT-4.1-mini	Oracle	137 (91.3%)	13 (8.7%)	0 (0%)	N/A
GPT-4.1	Oracle	140 (93.3%)	10 (6.7%)	0 (0%)	N/A
GPT-4.1-mini	10, No Meta	93 (62.0%)	49 (32.7%)	8 (5.3%)	0.032
GPT-4.1-mini	10, Naive RAG	98 (65.3%)	43 (28.6%)	9 (6%)	0.031
GPT-4.1-mini	10, Naive RAG, Planner	104 (69.3%)	41 (27.3%)	5 (3.3%)	0.029
GPT-4.1-mini	10, No Val	107 (71.3%)	42 (28.0%)	1 (0.7%)	0.028
GPT-4.1-mini	Mimir, 10	114 (76.0%)	34 (22.7%)	2 (1.3%)	0.026
GPT-4.1-mini	Mimir, 30	114 (76.0%)	31 (20.7%)	5 (3.3%)	0.026
GPT-4.1	10, Naive RAG, Planner	118 (78.7%)	31 (20.7%)	1 (0.7%)	0.127
o4-mini**	Mimir, 10	119 (79.3%)	28 (18.7%)	3 (2.0%)	0.069
GPT-4.1	Mimir, 10	124 (82.7%)	22 (14.7%)	4 (2.7%)	0.121

Table 9: Iteration 1: Retrieval performance across configurations.

Model	Setting	Hit@1	Hit@5	Avg docs	Avg chunks
GPT-4.1	Mimir, 10	0.67	0.87	2.02	8.19
GPT-4.1	10, Naive RAG, Planner	0.47	0.88	14.23	31.33
o4-mini	Mimir, 10	0.69	0.84	1.67	5.07
GPT-4.1-mini	Mimir, 10	0.65	0.81	1.59	4.75
GPT-4.1-mini	Mimir, 30	0.64	0.81	1.91	8.45
GPT-4.1-mini	10, No Val	0.66	0.85	2.49	20.07
GPT-4.1-mini	10, No Meta	0.53	0.73	2.09	4.57
GPT-4.1-mini	10, Naive RAG, Planner	0.37	0.79	11.16	22.16
GPT-4.1-mini	10, Naive RAG	0.17	0.75	5.53	9.99

5.6. Second Iteration

The benchmark methodology remained consistent with Iteration 1 (FinanceBench 150-question subset, Judge agent evaluation). This iteration aimed to improve two identified weaknesses.

1. **Calculator Tool for Writer agent:** To address arithmetic errors observed in numerical reasoning tasks. This was implemented using `sympy`, allowing the agent to symbolically parse mathematical formulas, substitute variables with their corresponding values, evaluate the expressions numerically, and obtain results rounded to a specified precision.
2. **Table-aware Chunking Strategy:** The Chonkie chunker was enhanced to better handle tables. After an initial recursive chunking based on Markdown structure (1800-character target, 300-character overlap), a specific table-merging step was introduced. This step identifies consecutive chunks where the first ends with a table row and the next begins with a table row (identified by lines starting with a pipe '|'). Such chunks are merged together, up to a maximum length of 3600 characters (twice the standard chunk size), to prevent tables from being split across multiple, disconnected chunks. In effect, the strategy preserves a larger contiguous table fragment so that later retrieval returns complete financial rows more often. The original overlap refinement step was then applied to these

potentially merged chunks.

These targeted changes aimed to improve accuracy, particularly in numerical and table-based questions. To analyze the impact of the table-aware chunking strategy introduced, we examined the distribution of chunks per report. Without table merging (as in Iteration 1), the mean number of chunks per report was 431.6 (standard deviation: 284.6). With table merging enabled, the mean decreased to 380.2 chunks per report (standard deviation: 253.0). This reduction indicates that the table-merging step effectively consolidated content, leading to fewer, potentially more contextually complete, chunks for table-heavy documents.

5.6.1. Performance of Models

Table 10 presents the performance of the improved Mimir-RAG. All configurations showed improvement. Notably, GPT-4.1 with 20 chunks achieved 89.3% accuracy, surpassing its Iteration 1 best (82.7%) and the original 85% FinanceBench Oracle. This is approaching our updated GPT-4.1 Oracle baseline of 93.3%, demonstrating the efficacy of the enhancements. The Naive RAG configuration (GPT-4.1-mini) also saw a performance boost of approximately 3 percentage points (from 65.3% to 68.0%) from the improved table-aware chunking (no calculator in the Naive approach).

GPT-4.1-mini also benefited significantly: its 10-chunk configuration accuracy rose from 76.0% (Iteration 1, full

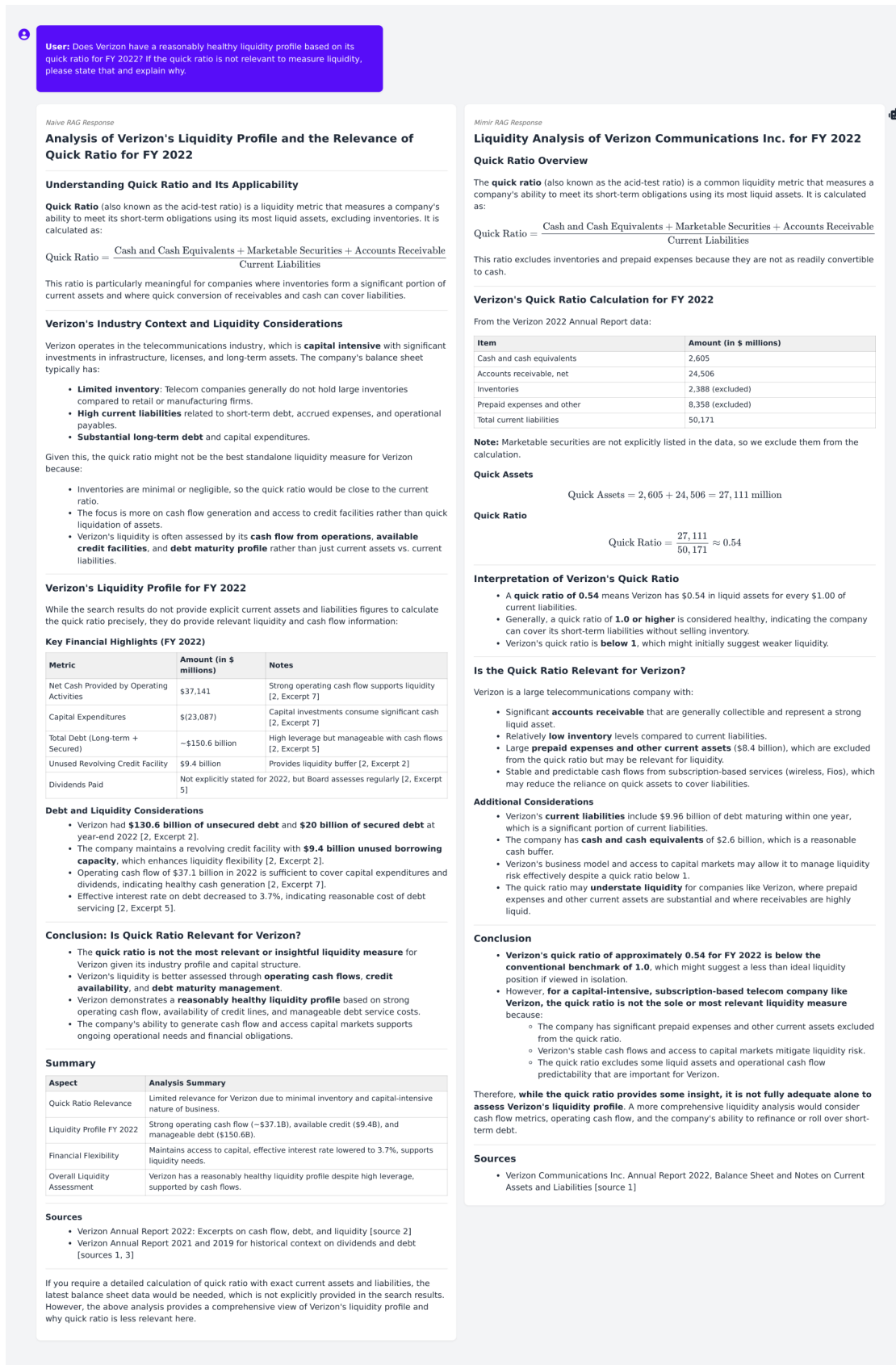


Figure 11: Output example comparing Naive RAG (left) with MimirRAG (right, using GPT-4.1-mini) for a question on Verizon’s FY 2022 Quick Ratio. Naive RAG fails to retrieve the correct data from multiple documents, while MimirRAG successfully identifies the correct document and computes the ratio.

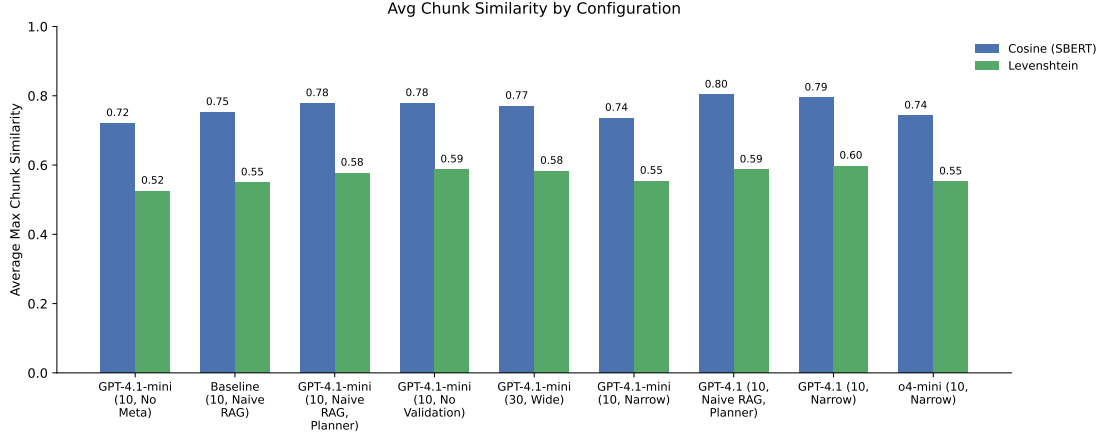


Figure 12: Iteration 1: Average chunk similarity (Cosine and Levenshtein).

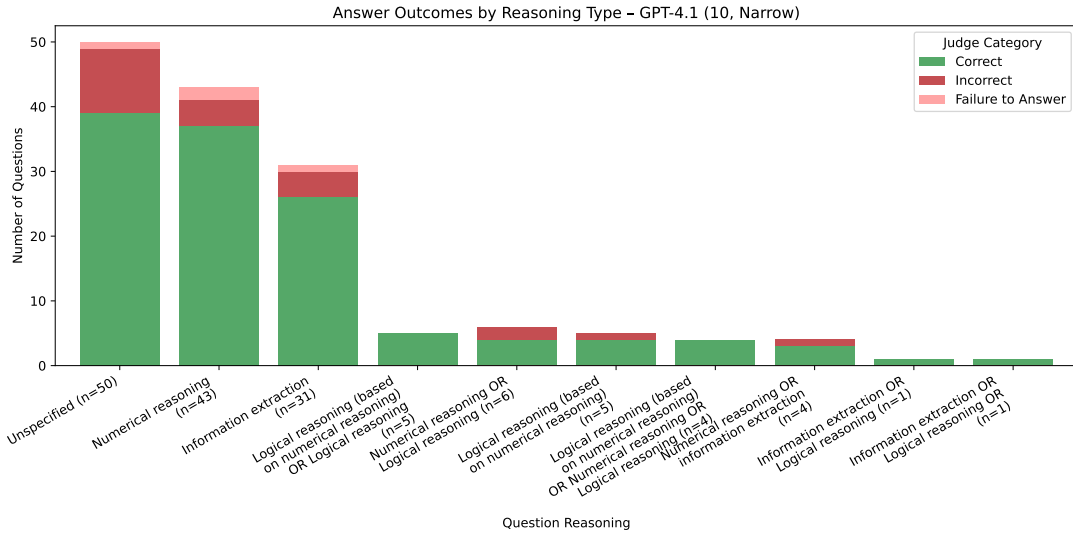


Figure 13: Iteration 1: Answer outcomes by reasoning type for GPT-4.1 (10).

MimirRAG) to 79.3% (Iteration 2, full MimirRAG) with the new chunking and calculator tool. Increasing the chunk depth to 20 in this improved system further boosted its accuracy to 84.7%, with a very low ‘Failed to Answer’ rate (0.7%). Unlike Iteration 1, scaling search breadth (chunk count) now yielded more substantial gains for GPT-4.1-mini.

5.6.2. Retrieval Analysis

At the document retrieval level, the core filtering and document search mechanisms remained consistent with Iteration 1, the results are shown in Table 11. At the document level, Hit@1 performance showed varied changes. For the GPT-4.1 model, increasing the retrieved chunks from 10 (Iteration 1, Hit@1: 0.67) to 20 (Iteration 2, Hit@1: 0.65) saw a slight decrease in Hit@1. For GPT-4.1-mini, comparing the 10-chunk configuration between Iteration 1 (Hit@1: 0.65) and Iteration 2 (Hit@1: 0.65) showed no change in Hit@1. However, within Iteration 2, increasing

the chunk depth for GPT-4.1-mini from 10 to 20 significantly improved Hit@1 from 0.65 to 0.73. This latter configuration also surpassed the Hit@1 achieved by GPT-4.1-mini with 30 chunks in Iteration 1 (0.64). The Hit@5 metric remained relatively stable or showed slight improvements for comparable MimirRAG configurations.

The average number of distinct documents processed per question (‘Avg docs’) varied across configurations, with some Iteration 2 setups (e.g., GPT-4.1-mini with 10 or 20 chunks) processing slightly more unique documents on average than their direct Iteration 1 counterparts. The average number of chunks sent to the Writer Agent (‘Avg chunks’) is now influenced by both the number of initially retrieved chunks and the effects of the table-aware chunking strategy, as discussed; some chunks are now larger and contain more information.

Figure 14 shows the chunk similarity scores for Iteration

Table 10: Iteration 2: Performance comparison with system improvements (table-aware chunking and calculator tool).

Model	Configuration	Correct	Incorrect	Failed to answer	Rel. Cost (\$/% Acc.)
GPT-4.1-mini	10, Naive RAG	102 (68%)	44 (29.3%)	4 (2.6%)	0.029
GPT-4.1-mini	Mimir, 10	119 (79.3%)	25 (16.7%)	6 (4.0%)	0.025
GPT-4.1-mini	Mimir, 20	127 (84.7%)	22 (14.7%)	1 (0.7%)	0.024
GPT-4.1	Mimir, 20	134 (89.3%)	15 (10.0%)	1 (0.7%)	0.112

Table 11: Iteration 2: Retrieval performance across configurations.

Model	Setting	Hit@1	Hit@5	Avg docs	Avg chunks
GPT-4.1	Mimir, 20	0.65	0.85	2.01	9.25
GPT-4.1-mini	Mimir, 20	0.73	0.86	1.89	7.53
GPT-4.1-mini	Mimir, 10	0.65	0.85	1.79	5.29
GPT-4.1-mini	Naive RAG	0.25	0.79	5.58	10.00

2. The best performing model, GPT-4.1 (Mimir, 20), achieved the highest average Cosine and Levenshtein similarity scores, suggesting that the retrieved chunks were semantically closer and more textually similar to the gold-standard evidence.

5.6.3. Error Analysis by Reasoning Type

An error analysis for the best-performing configuration in Iteration 2 (GPT-4.1, Mimir, 20) is presented in Table 12. The introduction of the calculator tool and table-aware chunking aimed to reduce errors, particularly in numerical and table-based questions. Interestingly, while overall accuracy improved significantly in Iteration 2, an analysis revealed that six questions which were answered correctly by the top configuration in Iteration 1 were *answered incorrectly in Iteration 2 by its respective top configuration*. This suggests that while the system improvements were broadly beneficial, they may have introduced regressions for a small subset of cases. For two of these, we see that this is due to different judgment from the Judge agent and not a difference in response from MimirRAG. This highlights a potential source of variance in automated evaluation. The remaining four regressions appear to be genuine changes in system output. Further investigation into these specific instances is warranted and will be discussed in Section 7.1. If these six questions had been answered correctly, the accuracy for GPT-4.1 (Mimir, 20) would have reached 93.3%.

5.6.4. Lessons Learned from second Iteration

Finally, we learned from second iteration: 1) Targeted improvements yield significant gains: The calculator tool and table-aware chunking directly addressed weaknesses from Iteration 1, leading to substantial accuracy increases across models. 2) Surpassing original Oracle baseline: MimirRAG (GPT-4.1, Mimir, 20), with 89.3% accuracy, surpassed the original 85% FinanceBench Oracle (GPT-4-Turbo) and closely approached our more up-to-date Oracle baseline (93.3% with GPT-4.1). 3) Increased chunk depth becomes beneficial: with better underlying processing (es-

pecially more complete table chunks), providing more context became more effective, unlike the diminishing returns seen in Iteration 1 for smaller models. 4) Reduced errors and failures: the ‘Incorrect’ and ‘Failed to Answer’ rates dropped, indicating more robust retrieval and downstream reasoning.

6. Expert Validation

This section details an iterative evaluation process, beginning with an initial prototype assessment by one expert (P1), leading to system refinements, followed by a broader evaluation with four financial analysts (P1-P4). The data collection process followed a mixed-method strategy i.e., qualitative data from interviews and open-ended survey responses were thematically coded (e.g., for workflow impact, trust calibration, feature requests), while quantitative data from Likert-scale surveys (A/B test in Iteration 1, feedback survey in Iteration 2) provided descriptive statistics on perceived system attributes.

6.1. Iteration 1: Formative Evaluation with Participant 1

The initial prototype was evaluated by one financial analyst (P1) through an A/B test against Microsoft Copilot and a semi-structured interview. Microsoft Copilot was selected as it is the current alternative at the bank and is based on GPT-4o. Table 13 presents the mean scores from the A/B test, where P1 evaluated MimirRAG and Copilot on five financial questions. MimirRAG outperformed Copilot across all five dimensions, notably in relevance (+1.0) and usefulness (+0.6) on a five-point Likert scale. Qualitative feedback from P1, gathered through the interview and A/B test comments, provided richer context. P1 found MimirRAG’s answers more comprehensive and historically aware than Copilot’s, and valued the chat interface for iterative questioning. While initial impressions of MimirRAG were positive, P1 emphasized that sustained trust would require consistent and accurate performance over time, given the potential impact of errors in their professional recommendations, and that he would

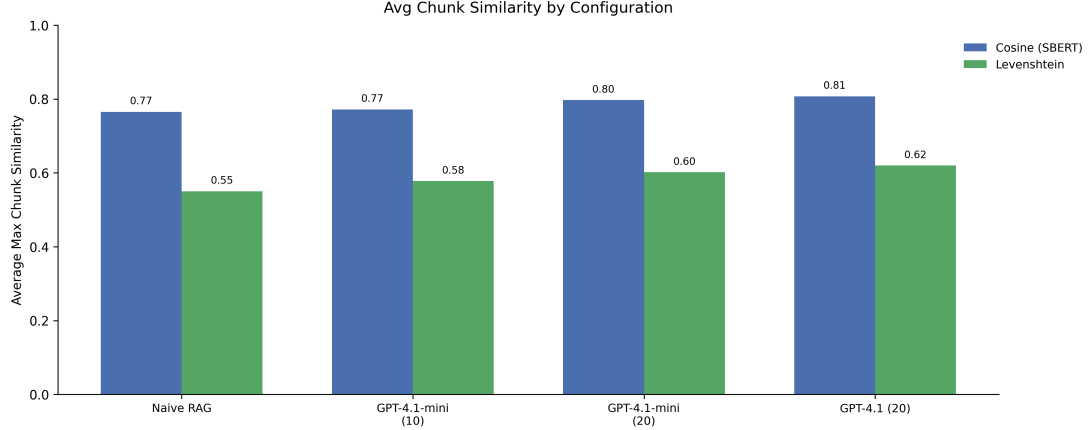


Figure 14: Iteration 2: Average chunk similarity (Cosine and Levenshtein).

Table 12: Iteration 2: Error breakdown by question type for GPT-4.1 (20).

Question Type	Correct	Failure to Answer	Incorrect
Domain-relevant	45	1	4
Metrics-generated	47	0	3
Novel-generated	42	0	8

Table 13: Average analyst ratings (1 = poor, 5 = excellent) (N=1)

Dimension	Copilot	MimirRAG
Relevance to query	3.2	4.2
Factual correctness	4.0	4.2
Usefulness	3.0	3.6
Clarity and reasoning	3.6	4.0
Source traceability	4.4	4.6

always review outputs manually rather than expecting a ‘plug-and-play’ solution.

The most actionable insight from this initial evaluation was P1’s strong emphasis on integrating personal research notes to create a ‘private knowledge vault’, identified as a key unique feature. This feedback, along with the generally positive reception and the noted importance of trust and consistency, guided key enhancements implemented before Iteration 2. First, a *Notes Integration Feature* was developed, providing a system for uploading, managing, and querying personal notes, which included processing P1’s own notes for testing. Second, to gather broader perspectives, the *number of evaluators was expanded to include three additional analysts* for the subsequent iteration.

6.2. Analysts Feedback Survey

Following system enhancements, the prototype was evaluated by four financial analysts (P1-P4). Feedback was gathered through a survey incorporating Likert-scale questions and open-ended responses, supplemented by semi-structured follow-up interviews with P2, P3, and P4. P1

also provided specific feedback on the newly integrated notes feature.

6.2.1. Quantitative Findings

The Likert-scale responses (where 1 = Worst/Least, 5 = Best/Most) provided the following insights (Table 14):

The system’s transparency was rated highly (Mean: 4.25), with good potential for time-saving (Mean: 4.00). Helpfulness and trust scored positively (Mean: 3.75 each). As elaborated in the qualitative findings, trust was noted as needing to be earned over consistent use.

6.2.2. Qualitative Findings

Qualitative insights (gathered from survey and interviews), synthesized from open-ended survey responses and follow-up interviews (P2-P4), and P1’s feedback on the notes feature, revealed several key themes. Users valued features like the Notes search (P1), ease of navigation (P2), qualitative data retrieval (P3), and system speed (P4). Key improvement areas identified included expanding data access (P1, P2), improving document-level control (P2), better handling of tables/graphs (P3), and enhanced output formats such as charts or spreadsheet integration (P4).

Willingness to integrate the tool into regular workflows varied: P4 was open, P1 and P3 were tentative, and P2 was hesitant, citing the richer feature sets in commercial tools. Regarding accuracy, P2 and P4 reported strong performance for factual queries and direct document retrieval with minimal hallucination. In contrast, P3 encountered difficulties with table and figure extraction, though text summarization worked well. Some participants mentioned

Table 14: Analyst Survey Ratings (N=4)

Question	P1	P2	P3	P4	Mean
3. How helpful is the system in retrieving info?	3	4	3	5	3.75
4. How clear/transparent is the reasoning?	4	4	4	5	4.25
5. To what extent do you trust the system?	4	3	4	4	3.75
6. Would the system potentially help you to save time?	4	4	3	5	4.00

reverting to other tools for certain tasks or when encountering perceived inconsistencies. Trust was consistently described as evolving and highly sensitive to performance; P4 considered the system trustworthy but fragile, while P3’s mixed experience led to caution, emphasizing that consistent functionality across all data types is essential, often leading to cautious initial use and manual verification of outputs. A recurring theme was the critical need for broader data integration (e.g., live news, financial, transcripts) and improved processing capabilities to meet professional expectations.

6.2.3. Way Forward

The Iteration 2 evaluation with four analysts confirmed the value of the personal notes integration (particularly enhancing usability for P1, who tested this feature). Overall feedback from surveys and interviews was positive regarding helpfulness, transparency, and time-saving potential. However, two key areas for future improvement emerged: 1) Trust, which hinges on consistent performance, especially with structured data. 2) The need for expanded data integration and processing capabilities to align with professional tools and expectations.

7. Hindsight

7.1. Interpretation of the Findings

7.1.1. Findings from Iteration 1

Ablation studies in Iteration 1 revealed that individual components of the MimirRAG pipeline significantly affect accuracy. Even the reimplemented Naive RAG using GPT-4.1-mini, combined with Docling’s PDF-to-Markdown conversion and structure-aware chunking, achieved a solid baseline accuracy of 65.3%. Introducing a Planner-based query expansion improved performance to 69.3%, with a notable increase in document retrieval accuracy (Hit@1 from 0.17 to 0.37), driven by better initial queries.

In contrast, the No Meta configuration—representing hierarchical search without metadata filtering—underperformed (62.0% accuracy), likely due to semantic ambiguity in document summaries. However, integrating hierarchical search with metadata filtering significantly boosted performance, raising accuracy to 76.0% and Hit@1 to 0.65. It also reduced the average number of retrieved documents (from 14.23 to 2.02), minimizing noise and improving precision. This highlights the critical

role of metadata in enhancing retrieval efficiency and addresses a key gap in existing financial RAG systems.

Iteration 1 showed that the Validator agent significantly reduced noise at the chunk level, improving accuracy by 4.7% (from 71.3% without Validator to 76.0% with it). Although the No Validator setup yielded slightly higher chunk similarity scores, it resulted in less accurate answers—indicating that the Validator’s strength lies in filtering out irrelevant or contradictory content, thus enhancing final answer quality beyond what similarity metrics alone can capture. Additionally, targeted model allocation proved beneficial: using the stronger o4-mini model for the Planner and Writer (while keeping GPT-4.1-mini for Search and Validation) yielded a 3% accuracy gain. Overall, results confirmed that system performance scales with LLM quality and highlighted the value of modular RAG design, allowing easy integration of more capable models in specific roles.

7.1.2. Findings from Iteration 2

During Iteration 2, MimirRAG continued to outperform baseline methods; for example, using GPT-4.1-mini, it achieved 84.7% accuracy (20 chunks) compared to Naive RAG’s 68.0% (Table 10). The advantage of stronger models remained clear, with GPT-4.1 reaching 89.3% accuracy, surpassing GPT-4.1-mini (84.7%). Iteration 2 addressed specific weaknesses from Iteration 1, including numerical reasoning and table fragmentation. The introduction of a calculator tool for the Writer Agent and a table-aware chunking strategy (‘Chonkie Chunker’) led to measurable improvements. Table-aware chunking alone improved Naive RAG performance by approximately 2.7%, increasing GPT-4.1-mini (10, Naive RAG) accuracy from 65.3% to 68.0%, even without the calculator tool.

A key insight was that expanding retrieval breadth became more beneficial in Iteration 2. While increasing from 10 to 30 chunks showed no gain in Iteration 1 (both at 76.0%, Table 8), raising chunks from 10 to 20 in Iteration 2 led to a significant jump for GPT-4.1-mini—from 79.3% to 84.7% (Table 10). This suggests that improved chunking—especially merged table segments—produced more useful retrievals because the Writer agent more often received complete numerical context rather than fragmented tables. However, these gains introduced new complexities. Six questions previously answered correctly in Iteration 1 were marked incorrect in Iteration 2’s top Mimir-

RAG configuration. In two cases, answers were nearly identical but received different judgments from the LLM-based Judge agent; the remaining four were true regressions. This reflects known issues in FinanceBench, where other researchers have also noted evaluation inconsistencies [46, 16]. These findings highlight the trade-offs in RAG system optimization and the limitations of relying solely on automated evaluation.

7.1.3. Findings from Expert Feedback

Our iterative expert validation with financial analysts underscored that successful RAG tool integration hinges on a triad of interconnected factors: earned trust, comprehensive data integration, and deep personalization. The critical importance of *trust calibration* [47] was evident. While MimirRAG’s low hallucination rates were valued and initial quantitative trust was moderate (Mean: 3.75/5), qualitative feedback across iterations revealed trust as an evolving state, earned through consistent and reliable performance. Analysts, aware of the high-stakes nature of their work, approached the system cautiously, often verifying results manually (a behavior P1 highlighted and others echoed). This cautious adoption and dynamic trust adjustment based on task-specific performance (e.g., P3’s issues with structured data) sometimes led to reverting to familiar tools, aligning with established trust models in automation [47, 48] where users avoid over-reliance until dependability is proven.

Next, the *scope and integration of data* significantly impacted perceived utility. While MimirRAG was useful with uploaded documents, analysts (notably P1, P2, P4) consistently stressed the need for broader, dynamic data sources and advanced numerical processing. The current static corpus was seen as limiting efficiency for real-time tasks, prompting the use of alternative platforms. This highlights a key challenge: future specialized RAG systems require architectural flexibility for robust, low-latency integration of heterogeneous live data, moving beyond curated knowledge bases to genuinely support dynamic professional data ecosystems.

Finally, the enthusiastic reception of the ‘Notes Integration’ feature, driven by P1’s desire for a ‘private knowledge vault’, demonstrated the critical role of *personalization*. For expert users, seamlessly querying and integrating their curated insights with system-retrieved information is a core value proposition, not just a convenience. This suggests that future RAG systems should prioritize architectures enabling secure and efficient personal knowledge base integration, offering a key differentiator by transforming them into customized analytical partners. In essence, developing effective RAG tools for professionals demands more than advanced retrieval. It requires a user-centric approach that holistically addresses the earned nature of trust, the necessity of comprehensive data integration, and the significant value of deep personalization to achieve gen-

uine workflow integration.

7.2. Implications of the Results

The findings of this research carry important implications for various stakeholders and the broader understanding of RAG in finance.

7.2.1. Practical Implications

1) For Developers & System Designers: Our findings validate specific design principles derived from MimirRAG: domain-appropriate, table-aware chunking, robust metadata filtering for targeted retrieval, layered agentic processes with validation, and user-centric design for trust and personalization. These are critical for building effective financial RAG tools. 2) For Financial Analysts: MimirRAG demonstrates a path to reduced information overload, enabling a shift from data gathering to higher-value interpretation, as suggested by their feedback on potential time savings and the utility of features like ‘Notes Integration’. 3) Empirical Validation of Metadata’s Crucial Role: This study provides strong empirical validation for metadata’s crucial role in enhancing RAG precision (e.g., Iteration 1 Hit@1 improvement from 0.53 to 0.65 with metadata), a principle highly generalizable to other domains with semi-structured corpora (e.g., legal, medical).

7.2.2. Theoretical Implications

This research advances the field of 1) Human-AI Collaboration by providing empirical insights into trust calibration in high-stakes professional settings. Analysts exhibited ‘rational under-utilization’, gradually building trust through consistent performance, a pattern likely applicable in other critical domains. 2) The study also empirically validates the Modular RAG paradigm in finance, demonstrating its advantages over simpler RAG setups through tailored, task-specific processing. Iterative improvements in the MimirRAG prototype highlight the power and adaptability of this modular approach. Although designed for financial analysts at a Danish bank, MimirRAG’s agentic, modular architecture, featuring enhanced query handling, targeted error mitigation, and tool integration suggests broad applicability. This design is especially suited for other specialized domains that require complex, multi-step retrieval and synthesis workflows, where generic RAG systems may be insufficient. Adapting this approach would require domain-specific tuning of agents and tools, but the core design philosophy most likely remains transferable.

7.2.3. Societal Implications and Considerations

1) Socio-Technical Shifts: Beyond the direct benefits to financial analysts, the deployment of advanced RAG systems like MimirRAG also asks for reflection on broader societal shifts. The increasing automation of information-intensive tasks may reshape the skillsets required in the financial labor market, potentially shifting human analysts

towards more strategic and interpretive roles while requiring ongoing adaptation and training. 2) Ethical Governance and Responsible AI: as AI tools become more essential in financial analysis, ensuring proper ethical governance and addressing potential biases inherent in models or data becomes important to maintain market integrity and public trust. While MimirRAG is designed to augment human expertise, the wider adoption of such technologies calls for a proactive approach to these societal and ethical dimensions, particularly with emerging regulations like the EU AI Act [49].

7.3. Limitations

While this research offers important insights into RAG-based solutions for financial analysts, its findings should be interpreted with consideration of limitations, specifically related to the benchmark scope, the nature of expert validation, and constraints of the prototype system.

At the document-processing level, an additional limitation is the imperfect recovery of structure from PDFs. Although our parsing stack preserves layout and tables where possible, OCR and table extraction can still introduce errors in scanned or visually complex filings, which in turn can affect chunking and downstream retrieval.

1. *Benchmark Experiments:* Several limitations should be considered when interpreting these findings. First, the evaluation is based on the 150-question open-source subset of the FinanceBench dataset. While useful for detailed analysis, results may not generalize to the full, non-public dataset, which we were unable to access. Additionally, our study relies on a single benchmark. Although other benchmarks like FinDER [50] emerged during the research, their datasets were not publicly available for comparison. We also identified data quality issues within the FinanceBench subset, such as missing information (e.g., company names, fiscal years), incorrect gold answer calculations, and ambiguity in questions that allow multiple valid answers depending on financial ratio conventions. These concerns have also been reported by others [16].
2. *OpenAI GPT-focused:* Our experiments were limited to LLMs from OpenAI. A broader investigation, including other proprietary models and a diverse range of open-source alternatives, could provide a more comprehensive understanding of how different model architectures and training methodologies impact RAG performance in the financial domain.
3. *Judge Agent:* The evaluation of system responses was conducted using an LLM-based Judge agent. Although this is an increasingly common practice in NLP research [51, 52] and we try to mitigate known self-bias problem [53, 54] by using a different model

(GPT-4o) for judging than those used in the RAG pipeline, LLM-based evaluation can have inherent limitations and may not perfectly replicate human judgment.

4. *Prompt engineering:* Our prompt engineering was largely ad hoc. While we iteratively refined prompts based on observed performance, we did not employ systematic prompt optimization frameworks such as DSPy [55], which could potentially unlock further performance gains through more structured prompt engineering.

In addition, the expert validation had several limitations. It involved a small (four) sample of financial analysts, all from a single case company, which limits the generalizability of findings on usability, trust, and workflow integration. Participant interactions with the MimirRAG prototype were relatively brief (e.g., two weeks for P1), potentially missing longer-term dynamics. The user interface prioritized functional evaluation over user experience design, which may have influenced perceptions, while systematically comparing different UI/UX (user experience) for financial RAG tools would yield valuable design guidelines. Additionally, specific features like ‘search-my-notes’ were only tested by one participant (P1), limiting broader conclusions about their effectiveness. Research should explore how the increasing abilities of AI tools (such as RAG systems) might reshape the core skill sets required of financial analysts, while further investigation is needed into which explainability and interpretability techniques are most effective and valued by financial analysts.

7.4. Lessons Learned

Beyond the implemented enhancements, Iteration 2 also explored additional improvements that were ultimately excluded from the final system due to limited performance benefits or added complexity on the benchmark dataset.

1. First, inspired by [56], a ‘think’ tool was tested for the Planner and Writer agents to support internal monologue and structured reasoning before complex tool use or generation. However, in our configuration, this addition did not improve performance. To preserve system simplicity, it was not included in the final Iteration 2 setup.
2. Second, we experimented with giving the Search agent finer control over retrieval by splitting the hybrid search into two tools: search reports and search chunks. The agent would first filter documents using search reports and then perform chunk-level retrieval using selected document IDs. Although this approach improved the document-level Hit@1 metric, it introduced latency and did not result in higher overall accuracy on FinanceBench. Therefore, the simpler unified search approach was retained.

While not included in the final system due to limited benchmark gains, the split search strategy may offer practical advantages in real-world scenarios. Unlike benchmarks, which ensure document availability, live systems may encounter missing documents (e.g., a SEC F-20 form for year 2023 filing). In such cases, a more autonomous Search agent, capable of adaptive strategies like expanding date ranges or checking alternate years, could improve retrieval. Although this behavior wasn't formally evaluated, it underscores the potential value of agent autonomy in production environments.

7.5. Ethical Aspect

Deploying powerful AI like RAG in high-stakes financial decision-making demands considering key ethical dimensions. Research is critically needed to investigate and mitigate potential biases embedded in the training data for both LLMs and financial embedding models, as these can affect RAG outputs, and to establish clear lines of accountability for any erroneous or biased results. Furthermore, developing robust frameworks for responsible AI governance and regulation within the financial industry is crucial. Such frameworks must address transparency, security, and data privacy and ensure compliance with emerging legislation, such as the European AI Act [49].

8. Conclusion

This work investigated the effective design and integration of RAG-based solutions for extracting meaningful financial insights, with a specific focus on enhancing the information retrieval workflows of financial analysts. Through the iterative design, development, and mixed-methods evaluation of our prototype system, MimirRAG, this research offers both practical architectural considerations and crucial insights into the human-centric factors affecting the adoption of such AI tools in the high-stakes financial domain. Our findings directly address our subresearch questions. MimirRAG's modular, multi-agent architecture (incorporating robust pre-retrieval, advanced agentic retrieval, and domain-specific generation) demonstrates high effectiveness for financial information search tasks (first subresearch question). From its evaluation, we derived key design principles (second subresearch question), including the critical use of domain-appropriate chunking, the significant impact of metadata integration, the benefits of layered retrieval with validation, and the necessity of designing for trust and seamless workflow integration. Furthermore, our expert validation demonstrates that successful adoption hinges on establishing *calibrated trust*, ensuring *comprehensive data integration*, and providing *personalized augmentation of expertise* (third subresearch question). These findings collectively suggest that meaningful financial insights can be effectively achieved when cutting-edge technology is combined with human-centric design. Such extraction extends beyond improved data retrieval,

requiring a system that processes complex financial data accurately, presents findings transparently, and supports iterative exploration. Through its demonstrated capabilities and positive expert feedback, MimirRAG serves as a compelling proof-of-concept that RAG technology holds significant promise for transforming information workflows in financial analysis.

To fully realize this potential, future development should prioritize the fine-tuning and evaluation of embedding models and re-rankers specifically on financial corpora, an important next step for improving domain-specific performance. Two key areas, identified through analyst feedback, stand out for enhancing tool value: first, integrating live data sources (news, full financial reports, transcripts) to expand the system's knowledge base, and second, enabling more natural, context-aware generation. The field would also benefit from new, comprehensive benchmarks reflecting realistic analyst workflows, supporting broader reasoning types and multi-document scenarios to better evaluate advanced RAG capabilities. In parallel with these technical advancements, ongoing research must continue to explore the human factors critical for successful adoption and integration of RAG systems into professional practice.

Acknowledgment

We sincerely thank the analysts who generously shared their time and expertise. Their candid feedback played a crucial role in shaping our experiments and reinforcing the validity of our conclusions.

Declarations

Use of Generative AI

Generative AI was used solely for proofreading (spelling, grammar, punctuation, and formatting issues) in this manuscript. It was not used for any conceptual, technical, or scholarly content, including figures, references, or code. All substantive work is the authors' own.

Ethical Approval

Not applicable.

Competing Interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Authors' Contributions

Magnus Methodology, Formal Analysis, Investigation, Software, Visualization, Writing - Original Draft. **Wilmer** Methodology, Formal Analysis, Investigation, Writing - Original Draft. **Somnath** Conceptualization, Supervision, Visualization, Writing - Review & Editing. **Mansoor** Conceptualization, Resources, Supervision - Review & Editing. **Mikkel** Conceptualization, Writing - Review & Editing.

Funding

Not applicable.

Availability of data and materials

Not applicable.

References

- [1] Y. Nie, Y. Kong, X. Dong, J. M. Mulvey, H. V. Poor, Q. Wen, S. Zohren, A survey of large language models for financial applications: Progress, prospects and challenges, arXiv preprint arXiv:2406.11903 (2024).
- [2] S. Ray, B. Srinivasan, Reducing hallucinations in large language models with custom intervention using amazon bedrock agents, amazon Web Services Blog, published November 26, 2024 (2024).
URL <https://aws.amazon.com/blogs/machine-learning/reducing-hallucinations-in-large-language-models-with-custom-intervention-using-amazon-bedrock-agents/>
- [3] B. Sarmah, D. Mehta, S. Pasquali, T. Zhu, Towards reducing hallucination in extracting information from financial reports using large language models, in: Proceedings of the Third International Conference on AI-ML Systems, 2023, pp. 1–5.
- [4] J. Impink, M. Paananen, A. Renders, Regulation-induced disclosures: evidence of information overload?, *Abacus* 58 (3) (2022) 432–478.
- [5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al., Retrieval-augmented generation for knowledge-intensive nlp tasks, *Advances in neural information processing systems* 33 (2020) 9459–9474.
- [6] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, H. Wang, H. Wang, Retrieval-augmented generation for large language models: A survey, arXiv preprint arXiv:2312.10997 2 (2023).
- [7] I. Iaroshev, R. Pillai, L. Vaglietti, T. Hanne, Evaluating retrieval-augmented generation models for financial report question and answering., *Applied Sciences* (2076-3417) 14 (20) (2024). doi:10.3390/app14209318.
URL <https://doi.org/10.3390/app14209318>
- [8] A. J. Yepes, Y. You, J. Milczek, S. Laverde, R. Li, Financial report chunking for effective retrieval augmented generation, arXiv preprint arXiv:2402.05131 (2024).
- [9] T. Zhou, P. Wang, Y. Wu, H. Yang, Finrobot: Ai agent for equity research and valuation with large language models, arXiv preprint arXiv:2411.08804 (2024).
- [10] P. Islam, A. Kannappan, D. Kiela, R. Qian, N. Scherrer, B. Vidgen, Financebench: A new benchmark for financial question answering, arXiv preprint arXiv:2311.11944 (2023).
- [11] J. Lee, M. Roh, Multi-reranker: Maximizing performance of retrieval-augmented generation in the financerag challenge, arXiv preprint arXiv:2411.16732 (2024).
- [12] S. Setty, H. Thakkar, A. Lee, E. Chung, N. Vidra, Improving retrieval for rag based question answering models on financial documents, arXiv preprint arXiv:2404.07221 (2024).
- [13] Q. Leng, J. Portes, S. Havens, M. A. Zaharia, M. Carbin, Long context rag performance of large language models, ArXiv abs/2411.03538 (2024).
URL <https://api.semanticscholar.org/CorpusId:273849918>
- [14] M. Rafiq, How ragie outperformed the financebench test, <https://www.ragie.ai/blog/ragie-outperformed-financebench>, blog post (Oct 2024).
- [15] M. Rafiq, How ragie outperformed the financebench test — part 2, <https://www.ragie.ai/blog/how-ragie-outperformed-the-financebench-test-part-2>, blog post, Part 2 (Nov 2024).
- [16] V. AI, Financebench performance of mafin 2.5, <https://github.com/VectifyAI/Mafin2.5-FinanceBench>, gitHub repository (2025).
- [17] D. Araci, Finbert: Financial sentiment analysis with pre-trained language models, arXiv preprint arXiv:1908.10063 (2019).
- [18] S. Wu, O. Irsoy, S. Lu, V. Dabrovolski, M. Dredze, S. Gehrmann, P. Kambadur, D. Rosenberg, G. Mann, Bloomberggpt: A large language model for finance (2023). arXiv:2303.17564.
URL <https://arxiv.org/abs/2303.17564>
- [19] X.-Y. Liu, G. Wang, H. Yang, D. Zha, Fingpt: Democratizing internet-scale data for financial large language models, arXiv preprint arXiv:2307.10485 (2023).
- [20] H. Yang, B. Zhang, N. Wang, C. Guo, X. Zhang, L. Lin, J. Wang, T. Zhou, M. Guan, R. Zhang, et al., Finrobot: an open-source ai agent platform for financial applications using large language models, arXiv preprint arXiv:2405.14767 (2024).
- [21] S. Shah, S. Ryali, R. Venkatesh, Multi-document financial question answering using llms, arXiv preprint arXiv:2411.07264 (2024).

- [22] B. Sarmah, D. Mehta, B. Hall, R. Rao, S. Patel, S. Pasquali, Hybridrag: Integrating knowledge graphs and vector retrieval augmented generation for efficient information extraction, in: Proceedings of the 5th ACM International Conference on AI in Finance, 2024, pp. 608–616.
- [23] X. Li, Z. Li, C. Shi, Y. Xu, Q. Du, M. Tan, J. Huang, W. Lin, Alphafin: Benchmarking financial analysis with retrieval-augmented stock-chain framework, arXiv preprint arXiv:2403.12582 (2024).
- [24] S. Kim, H. Song, H. Seo, H. Kim, Optimizing retrieval strategies for financial question answering documents in retrieval-augmented generation systems, arXiv preprint arXiv:2503.15191 (2025).
- [25] V. D. Lai, M. Krumdick, C. Lovering, V. Reddy, C. Schmidt, C. Tanner, Sec-qa: A systematic evaluation corpus for financial qa, arXiv preprint arXiv:2406.14394 (2024).
- [26] F. Taghvaei, A. Elahi, Combining financial data and news articles for stock price movement prediction using large language models, 2024 IEEE International Conference on Big Data (BigData) (2024) 4875–4883. URL <https://api.semanticscholar.org/CorpusId:273812246>
- [27] A. Singh, A. Ehtesham, S. Kumar, T. T. Khoei, Agentic retrieval-augmented generation: A survey on agentic rag, arXiv preprint arXiv:2501.09136 (2025).
- [28] S. Chen, L. Zeng, A. Raghunathan, F. Huang, T. C. Kim, Moa is all you need: Building llm research team using mixture of agents, arXiv preprint arXiv:2409.07487 (2024).
- [29] M. Poliakov, N. Shvai, Multi-meta-rag: Improving rag for multi-hop queries using database filtering with llm-extracted metadata, in: International Conference on Information and Communication Technologies in Education, Research, and Industrial Applications, Springer, 2024, pp. 334–342.
- [30] M. Saunders, P. Lewis, A. Thornhill, Research Methods for Business Students, 9th Edition, Pearson Education, 2023.
- [31] M. K. Sein, O. Henfridsson, S. Purao, M. Rossi, R. Lindgren, Action design research, MIS quarterly (2011) 37–56.
- [32] C. Auer, M. Lysak, A. Nassar, M. Dolfi, N. Livathinos, P. Vagenas, C. B. Ramis, M. Omenetti, F. Lindlbauer, K. Dinkla, et al., Docling technical report, arXiv preprint arXiv:2408.09869 (2024).
- [33] B. Pfizmann, C. Auer, M. Dolfi, A. S. Nassar, P. Staar, Doclaynet: A large human-annotated dataset for document-layout segmentation, in: Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining, 2022, pp. 3743–3751.
- [34] A. Nassar, N. Livathinos, M. Lysak, P. Staar, Tableformer: Table structure understanding with transformers, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 4614–4623.
- [35] L. Merrick, D. Xu, G. Nuti, D. Campos, Arctic-embed: Scalable, efficient, and accurate text embedding models, arXiv preprint arXiv:2405.05374 (2024).
- [36] P. Yu, L. Merrick, G. Nuti, D. Campos, Arctic-embed 2.0: Multilingual retrieval without compromise, arXiv preprint arXiv:2412.04506 (2024).
- [37] K. Enevoldsen, et al., Mmteb: Massive multilingual text embedding benchmark, arXiv preprint arXiv:2502.13595 (2025). doi:10.48550/arXiv.2502.13595. URL <https://arxiv.org/abs/2502.13595>
- [38] Y. Tang, Y. Yang, Finmteb: Finance massive text embedding benchmark, arXiv preprint arXiv:2502.10990 (2025).
- [39] L. E. Erdogan, N. Lee, S. Kim, S. Moon, H. Furuta, G. Anumanchipalli, K. Keutzer, A. Gholami, Plan-and-act: Improving planning of agents for long-horizon tasks, arXiv preprint arXiv:2503.09572 (2025).
- [40] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, E.-P. Lim, Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models, in: A. Rogers, J. Boyd-Graber, N. Okazaki (Eds.), Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Toronto, Canada, 2023, pp. 2609–2634. doi:10.18653/v1/2023.acl-long.147. URL <https://aclanthology.org/2023.acl-long.147/>
- [41] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, React: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations (ICLR), 2023.
- [42] S. Colvin, et al., Pydantic (Apr. 2025). doi:10.5281/zenodo.15174950.
- [43] X. Tan, H. Wang, X. Qiu, Y. Cheng, Y. Xu, W. Chu, Y. Qi, Struct-x: Enhancing large language models reasoning with structured data, arXiv preprint arXiv:2407.12522 (2024).

- [44] Anthropic, Building effective ai agents (Dec 2024).
URL <https://www.anthropic.com/engineering/building-effective-agents>
- [45] B. Zhang, How we build effective agents, YouTube, presented at the AI Engineer Summit 2025, New York. [Accessed: May 14, 2025] (2025).
URL https://www.youtube.com/watch?v=051042_2p5E
- [46] Z. Nguyen, A. Annunziata, V. Luong, S. Dinh, Q. Le, A. H. Ha, C. Le, H. A. Phan, S. Raghavan, C. Nguyen, Enhancing q&a with domain-specific fine-tuning and iterative reasoning: A comparative study, arXiv preprint arXiv:2404.11792 (2024).
- [47] J. D. Lee, K. A. See, Trust in automation: Designing for appropriate reliance, *Human factors* 46 (1) (2004) 50–80.
- [48] B. Shneiderman, Human-centered artificial intelligence: Reliable, safe & trustworthy, *International Journal of Human-Computer Interaction* 36 (6) (2020) 495–504.
- [49] European Parliament, EU AI Act: first regulation on artificial intelligence (2025).
URL <https://www.europarl.europa.eu/topics/en/article/20230601ST093804/eu-ai-act-first-regulation-on-artificial-intelligence>
- [50] C. Choi, J. Kwon, J. Ha, H. Choi, C. Kim, Y. Lee, J.-y. Sohn, A. Lopez-Lira, Finder: Financial dataset for question answering and evaluating retrieval-augmented generation, arXiv preprint arXiv:2504.15800 (2025).
- [51] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, H. Zhang, J. E. Gonzalez, I. Stoica, Judging llm-as-a-judge with mt-bench and chatbot arena, *ArXiv abs/2306.05685* (2023).
URL <https://arxiv.org/pdf/2306.05685.pdf>
- [52] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, W. Li, Y. Shen, S. Ma, H. Liu, et al., A survey on llm-as-a-judge, arXiv preprint arXiv:2411.15594 (2024).
- [53] W. Xu, G. Zhu, X. Zhao, L. Pan, L. Li, W. Wang, Pride and prejudice: Llm amplifies self-bias in self-refinement, in: *Annual Meeting of the Association for Computational Linguistics*, 2024.
URL <https://arxiv.org/pdf/2402.11436.pdf>
- [54] J. Ye, Y. Wang, Y. Huang, D. Chen, Q. Zhang, N. Moniz, T. Gao, W. Geyer, C. Huang, P.-Y. Chen, N. V. Chawla, X. Zhang, Justice or prejudice? quantifying biases in llm-as-a-judge, arXiv preprint arXiv:2410.02736 (2024).
- [55] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, et al., Dspy: Compiling declarative language model calls into self-improving pipelines, arXiv preprint arXiv:2310.03714 (2023).
- [56] Anthropic, The “think” tool: Enabling claude to stop and think (Mar 2025).
URL <https://www.anthropic.com/engineering/claude-think-tool>